

Analyzing Daily CO₂ Emissions Across Countries: Anomaly Detection, Structural Breaks, and Clustering

- **Group members:** Francesco Stellato, Daniele Cavuto, Parisa Izadiyar, Aashi Sudhanshu Mehrotra
- **Course:** Data mining & Machine learning
- **Instructors:** Giuseppe Longo & Roberta Siciliano
- **Date:** October 2025

Abstract: This project analyzes daily CO₂ emissions across multiple countries using a combination of data mining and machine learning techniques. After preprocessing and normalization based on population and GDP, the study explores emission patterns through visualization, statistical decomposition, and symbolic representation using SAX. Anomaly detection identifies rare temporal behaviors, while structural break analysis (CUSUM) confirms major disruptions around early 2020 due to the COVID-19 pandemic. Clustering methods (K-Means and Hierarchical) reveal groups of countries with similar emission dynamics or magnitude levels. Finally, LSTM neural networks are applied to forecast short-term emission trends, achieving low error rates and demonstrating the ability of deep models to capture complex temporal dependencies. Overall, the project provides a multi-level understanding of global CO₂ dynamics, integrating descriptive, analytical, and predictive approaches.

Table of Contents

- 1. Introduction
- 2. Data Preparation
 - 2.1 Dataset Overview
 - 2.2 Dataset extension and normalization
 - 2.3 Missing data and duplicate
- 3. Data Visualization
- 4. Symbolic Aggregate approXimation (SAX)
- 5. Anomaly Detection
- 6. Structural Break Detection (CUSUM)
- 7. Clustering Analysis
 - 7.1 Clustering analysis based on temporal evolution of emissions

- 7.2 Clustering analysis based on emissions magnitude
- 8. LSTM Forecasting
- 9. General Conclusions and Future Work

1. Introduction

Global CO₂ emissions are a key indicator of economic activity and environmental impact. Since the COVID-19 pandemic, geopolitical tensions, and energy crises have significantly affected emissions, analyzing their daily evolution can reveal important structural patterns and anomalies.

This project investigates how emission dynamics vary across countries, aiming to detect unusual patterns, identify structural breaks, and group nations with similar behaviors through clustering techniques. In addition to the descriptive analysis, we also implemented a Long Short-Term Memory (LSTM) neural network to forecast future emission trends, both at the global level and for selected countries, providing a predictive perspective that complements the exploratory insights.

Overall, the study combines data mining, machine learning, and time-series forecasting techniques.

Libraries

We use a pragmatic stack tailored to daily CO₂-emissions analysis (time series + sector & country breakdowns), anomaly detection, structural breaks, clustering, and reporting

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import set_config
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.pipeline import Pipeline
from sklearn.metrics import silhouette_score
import scipy as sp
import statsmodels.api as sm
import statsmodels.tsa.api as tsa
from scipy.stats import norm
from sklearn.neighbors import NearestNeighbors
from sklearn.decomposition import PCA
from sklearn.cluster import DBSCAN
from statsmodels.tsa.seasonal import seasonal_decompose
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, mean_absolute_percentage_error
import matplotlib.dates as mdates
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
import geopandas as gpd
```

```
from dotenv import load_dotenv
from tqdm import tqdm
```

2. Data preparation

The dataset used in this project comes from Carbon Monitor — an open, continuously updated initiative that provides near-real-time daily estimates of CO₂ emissions across multiple countries and economic sectors.

We downloaded the data from the official Carbon Monitor portal (<https://carbonmonitor.org/>).

2.1 Dataset overview

```
In [ ]: df = pd.read_csv('carbonmonitor-global_datas_2025-10-07.csv')
df.head()
```

```
Out[ ]:
```

	country	date	sector	value	Unnamed: 4
0	Brazil	01/01/2019	Industry	0.216618	NaN
1	Brazil	01/01/2019	Ground Transport	0.215540	NaN
2	Brazil	01/01/2019	International Aviation	0.020050	NaN
3	Brazil	01/01/2019	Domestic Aviation	0.019680	NaN
4	Brazil	01/01/2019	Residential	0.110618	NaN

It's better to remove unnecessary colum "Unnamed".

```
In [ ]: df = df.drop(columns=[col for col in df.columns if 'Unnamed' in col])
df.head()
```

```
Out[ ]:
```

	country	date	sector	value
0	Brazil	01/01/2019	Industry	0.216618
1	Brazil	01/01/2019	Ground Transport	0.215540
2	Brazil	01/01/2019	International Aviation	0.020050
3	Brazil	01/01/2019	Domestic Aviation	0.019680
4	Brazil	01/01/2019	Residential	0.110618

Dataset info

```
In [ ]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 204540 entries, 0 to 204539
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   country     204540 non-null  object
1   date        204540 non-null  object
2   sector      204540 non-null  object
3   value       204540 non-null  float64
dtypes: float64(1), object(3)
memory usage: 6.2+ MB

```

The dataset contains 204,540 observations and 4 variables.

Each row represents the daily CO₂ emissions (in million tonnes of CO₂ per day) produced by a specific sector in a given country on a particular date. The dataset covers multiple countries and economic sectors over several years, providing a detailed view of sectoral contributions to national emissions.

- country: name of the country (string)
- date: date of observation (string, later convertible to datetime format)
- sector: economic sector responsible for emissions (e.g., Industry, Ground Transport, Residential, International Aviation, etc.)
- value: daily CO₂ emissions for that sector and country in MtCO₂ (float)

All variables are complete (no missing values). The dataset can be aggregated by country, sector, or date to analyze emission trends, sectoral contributions, or global dynamics.

```

In [ ]: print("Number of countries: " + str(df['country'].unique()))
        df['country'].unique()

```

Number of countries: 14

```

Out[ ]: array(['Brazil', 'China', 'EU27 & UK', 'France', 'Germany', 'India',
              'Italy', 'Japan', 'ROW', 'Russia', 'Spain', 'United Kingdom',
              'United States', 'WORLD'], dtype=object)

```

```

In [ ]: df['sector'].unique()

```

```

Out[ ]: array(['Industry', 'Ground Transport', 'International Aviation',
              'Domestic Aviation', 'Residential', 'Power'], dtype=object)

```

```

In [ ]: df['date'] = pd.to_datetime(df['date'], dayfirst=True)
        df['date'].min(), df['date'].max()

```

```

Out[ ]: (Timestamp('2019-01-01 00:00:00'), Timestamp('2025-08-31 00:00:00'))

```

```

In [ ]: df.describe()

```

Out[]:		date	value
count		204540	204540.000000
mean	2022-05-02 00:00:00		2.385814
min	2019-01-01 00:00:00		0.000150
25%	2020-08-31 00:00:00		0.075188
50%	2022-05-02 00:00:00		0.315887
75%	2024-01-01 00:00:00		1.692845
max	2025-08-31 00:00:00		43.029100
std		NaN	5.900052

The dataset includes 204,540 observations with dates ranging from January 1 2019 to August 31 2025, providing a long-term view of daily CO₂ emissions. The variable value (daily emissions in MtCO₂/day) shows a wide range — from nearly zero up to about 43 MtCO₂/day, with a mean of 2.39 and a high standard deviation (≈ 5.9), indicating substantial variability across countries and sectors. The median (0.32 MtCO₂/day) is much lower than the mean, suggesting a right-skewed distribution, where most emissions are small but a few sectors or countries produce extremely large values.

2.2 Dataset extension and normalization

To make cross-country comparisons meaningful, we construct a new dataset that expresses CO₂ emissions relative to key socioeconomic indicators — population and gross domestic product (GDP).

Raw emission values (in MtCO₂/day) vary greatly among countries due to differences in size, population, and economic activity.

For instance, China's total emissions are much higher than Italy's, but that does not necessarily mean Chinese citizens or industries are more emission-intensive.

Therefore, normalization allows us to compare countries on a fairer scale.

Data sources

Population and GDP data are retrieved automatically from the World Bank Open Data API (via the `wbgapi` Python package).

- *Population* → indicator SP.POP.TOTL
- *GDP (current US\$)* → indicator NY.GDP.MKTP.CD

```
In [ ]: %pip install wbgapi
```

Collecting wbgapi

Downloading wbgapi-1.0.12-py3-none-any.whl.metadata (13 kB)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from wbgapi) (2.32.4)

Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from wbgapi) (6.0.3)

Requirement already satisfied: tabulate in /usr/local/lib/python3.12/dist-packages (from wbgapi) (0.9.0)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->wbgapi) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->wbgapi) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->wbgapi) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->wbgapi) (2025.10.5)

Downloading wbgapi-1.0.12-py3-none-any.whl (36 kB)

Installing collected packages: wbgapi

Successfully installed wbgapi-1.0.12

In []: `import wbgapi as wb`

```
def fetch_pop_gdp(countries_iso3, years_range):
    """
    Fetch total population and nominal GDP (current USD)
    for a list of ISO3 countries and a given year range.
    Returns a long DataFrame with: country, year, population, gdp.
    """
    POP = "SP.POP.TOTL"
    GDP = "NY.GDP.MKTP.CD"

    # Download data from the World Bank in wide format (countries x years)
    pop_wide = wb.data.DataFrame(POP, economy=countries_iso3,
                                time=years_range, labels=False, numericTimeKeys=True)
    gdp_wide = wb.data.DataFrame(GDP, economy=countries_iso3,
                                time=years_range, labels=False, numericTimeKeys=True)

    # Reshape population data from wide to long format
    pop_long = (
        pop_wide.stack()
        .rename('population')
        .reset_index()
        .rename(columns={'economy': 'country', 'level_1': 'year'})
    )
    # Reshape GDP data from wide to long format
    gdp_long = (
        gdp_wide.stack()
        .rename('gdp')
        .reset_index()
        .rename(columns={'economy': 'country', 'level_1': 'year'})
    )

    # Convert year to integer for consistency
    pop_long['year'] = pop_long['year'].astype(int)
    gdp_long['year'] = gdp_long['year'].astype(int)

    # Merge population and GDP tables on country and year
    out = pd.merge(pop_long, gdp_long, on=['country', 'year'], how='outer')
    # Return final tidy DataFrame
    return out
```

```

countries = ["BRA", "CHN", "FRA", "DEU", "IND", "ITA", "JPN", "RUS", "ESP", "GBR", "USA"]
years = range(2019, 2025)
# Fetch and display population and GDP data
pop_gdp_df = fetch_pop_gdp(countries, years)
print(pop_gdp_df.head())

```

	country	year	population	gdp
0	BRA	2019	207455459.0	1.873288e+12
1	BRA	2020	208660842.0	1.476107e+12
2	BRA	2021	209550294.0	1.670647e+12
3	BRA	2022	210306415.0	1.951924e+12
4	BRA	2023	211140729.0	2.191132e+12

Methodology

1. For each country and year, we merge the Carbon Monitor data with World Bank population and GDP figures.
2. We exclude aggregated entities not directly available in the World Bank database (WORLD, ROW, and EU27 & UK).
3. We compute normalized indicators:
 - CO₂ per capita (tCO₂/person/day):
 - CO₂ per GDP (tCO₂ per million USD/day):
4. We also aggregate the data at the annual level to compute:
 - tCO₂_per_capita_year
 - tCO₂_per_million_USD_year

```

In [ ]: # CREATE NORMALIZED DATAFRAMES (daily and annual)

# Map country names to ISO3 codes (for World Bank alignment)
name_to_iso3 = {
    "Brazil": "BRA",
    "China": "CHN",
    "France": "FRA",
    "Germany": "DEU",
    "India": "IND",
    "Italy": "ITA",
    "Japan": "JPN",
    "Russia": "RUS",
    "Spain": "ESP",
    "United Kingdom": "GBR",
    "United States": "USA",
}

# Copy main emissions dataframe
df_norm = df.copy()

# Parse dates and extract the year
df_norm["date"] = pd.to_datetime(df_norm["date"], dayfirst=True, errors="coerce")
df_norm["year"] = df_norm["date"].dt.year

# Remove aggregate entities not matched with World Bank data

```

```

exclude = {"WORLD", "ROW", "EU27 & UK"}
df_norm = df_norm[~df_norm["country"].isin(exclude)].copy()

# Add ISO3 country codes
df_norm["country_iso3"] = df_norm["country"].map(name_to_iso3).fillna(df_norm["c

# Merge with World Bank population and GDP data
df_norm = df_norm.merge(
    pop_gdp_df,
    left_on=["country_iso3", "year"],
    right_on=["country", "year"],
    how="left",
    suffixes=("", "_wb")
)

# Rename and clean redundant columns
df_norm = df_norm.rename(columns={'country': 'country_name'})
if 'country_wb' in df_norm.columns:
    df_norm = df_norm.drop(columns=['country_wb'])

# DAILY NORMALIZATIONS

# Convert emissions from MtCO2 to tons
df_norm["tCO2_day"] = df_norm["value"] * 1e6

# Per-capita emissions (tons per person per day)
df_norm["tCO2_per_capita_day"] = df_norm["tCO2_day"] / df_norm["population"]

# Emissions relative to GDP (tons per million USD per day)
df_norm["tCO2_per_million_USD_day"] = df_norm["tCO2_day"] / (df_norm["gdp"] / 1e

# ANNUAL NORMALIZATIONS (aggregated by country)

annual_df = (
    df_norm.groupby(["country_iso3", "country_name", "year"], as_index=False)
        .agg(MtCO2_per_year=("value", "sum"),
            population=("population", "first"),
            gdp=("gdp", "first"))
)

# Convert to tons and compute annual normalized metrics
annual_df["tCO2_per_capita_year"] = (annual_df["MtCO2_per_year"] * 1e6) / annual
annual_df["tCO2_per_million_USD_year"] = (annual_df["MtCO2_per_year"] * 1e6) / (

# Sort both dataframes for consistency
df_norm = df_norm.sort_values(["country_iso3", "date"]).reset_index(drop=True)
annual_df = annual_df.sort_values(["country_iso3", "year"]).reset_index(drop=Tru

#CONTROL OUTPUT
print("Daily normalized emissions:")
print(df_norm[["country_name", "date", "value", "tCO2_per_capita_day", "tCO2_per

print("\n Annual normalized emissions:")
print(annual_df[["country_name", "year", "tCO2_per_capita_year", "tCO2_per_milli

```

Daily normalized emissions:

	country_name	date	value	tCO2_per_capita_day \
0	Brazil	2019-01-01	0.216618	0.001044
1	Brazil	2019-01-01	0.215540	0.001039
2	Brazil	2019-01-01	0.020050	0.000097
3	Brazil	2019-01-01	0.019680	0.000095
4	Brazil	2019-01-01	0.110618	0.000533

	tCO2_per_million_USD_day
0	0.115635
1	0.115060
2	0.010703
3	0.010505
4	0.059050

Annual normalized emissions:

	country_name	year	tCO2_per_capita_year	tCO2_per_million_USD_year
0	Brazil	2019	1.911905	211.732037
1	Brazil	2020	1.800919	254.575877
2	Brazil	2021	2.015857	252.850198
3	Brazil	2022	1.923285	207.220809
4	Brazil	2023	1.899444	183.033259

Purpose

This normalization step provides two complementary perspectives:

- Per capita emissions used to highlight differences in individual or lifestyle carbon intensity.
- Per GDP emissions used to measure economic efficiency and sustainability (how much CO₂ is emitted per unit of economic output).

2.3 Missing data and duplicate

```
In [ ]: print("Missing Values:")
        print(df.isnull().sum())
        print('\nDuplicates based on Date, Country & Sector')
        df.duplicated(subset=['date', 'country', 'sector']).sum()
```

Missing Values:

```
country    0
date       0
sector     0
value      0
dtype: int64
```

Duplicates based on Date, Country & Sector

```
Out[ ]: np.int64(0)
```

There are no missing values and duplicated rows. It's necessary to check missing dates.

```
In [ ]: full_range = pd.date_range(start=df['date'].min(), end=df['date'].max())
        missing_dates = full_range.difference(df['date'].unique())
        print(f"Missing days in the dataset: {len(missing_dates)}")
```

Missing days in the dataset: 0

The first check shows that there are no missing dates globally, meaning that the time coverage is continuous from the first to the last recorded day.

However, even if the global date range is complete, some individual country–sector combinations could still have missing days (for example, if data for a given sector were not reported on certain dates). For this reason, a second check was performed at the (country, sector) level.

```
In [ ]: from tqdm import tqdm

missing_by_group = {}

for (country, sector), group in tqdm(df.groupby(['country', 'sector'])):
    group_dates = pd.to_datetime(group['date'])
    full_range = pd.date_range(start=group_dates.min(), end=group_dates.max())
    missing = full_range.difference(group_dates)
    if len(missing) > 0:
        missing_by_group[(country, sector)] = len(missing)

print(f"Number of (country, sector) pairs with missing dates: {len(missing_by_group)}")
```

100%|██████████| 84/84 [00:00<00:00, 159.34it/s]
Number of (country, sector) pairs with missing dates: 0

This confirms that each country–sector pair has complete daily data with no temporal gaps throughout the entire period.

This ensures that the dataset can be reliably used for time series analysis and temporal aggregation without requiring interpolation or imputation.

3. Data visualization

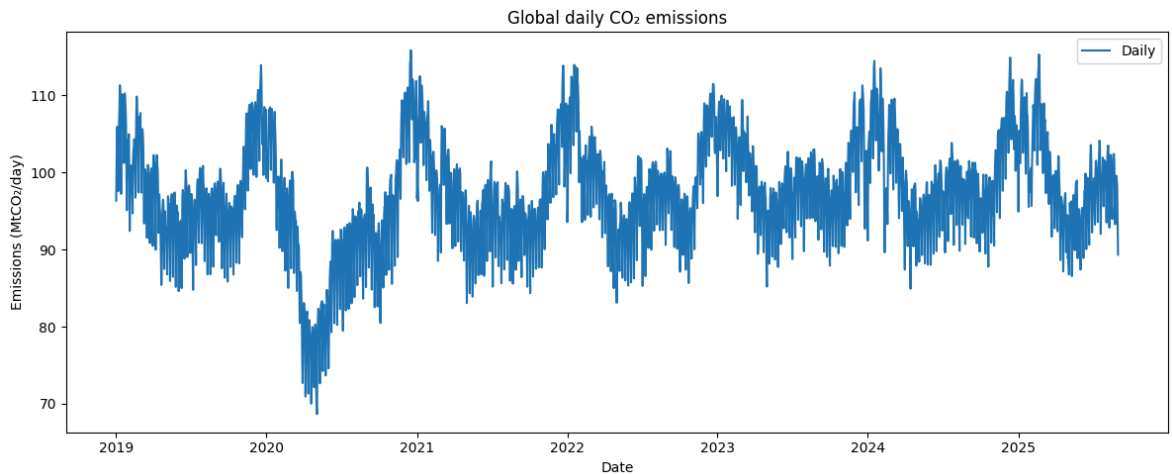
Global emissions

```
In [ ]: # Select only the global total (WORLD) data
df_world = df[df['country'] == 'WORLD']

# Aggregate daily emissions across all sectors
world_daily = df_world.groupby('date', as_index=False)['value'].sum()

# Plot global daily CO2 emissions over time
plt.figure(figsize=(12, 5))
plt.plot(world_daily['date'], world_daily['value'], label='Daily')

# Set plot title and axes
plt.title('Global daily CO2 emissions')
plt.xlabel('Date')
plt.ylabel('Emissions (MtCO2/day)')
plt.legend()
plt.tight_layout()
plt.show()
```



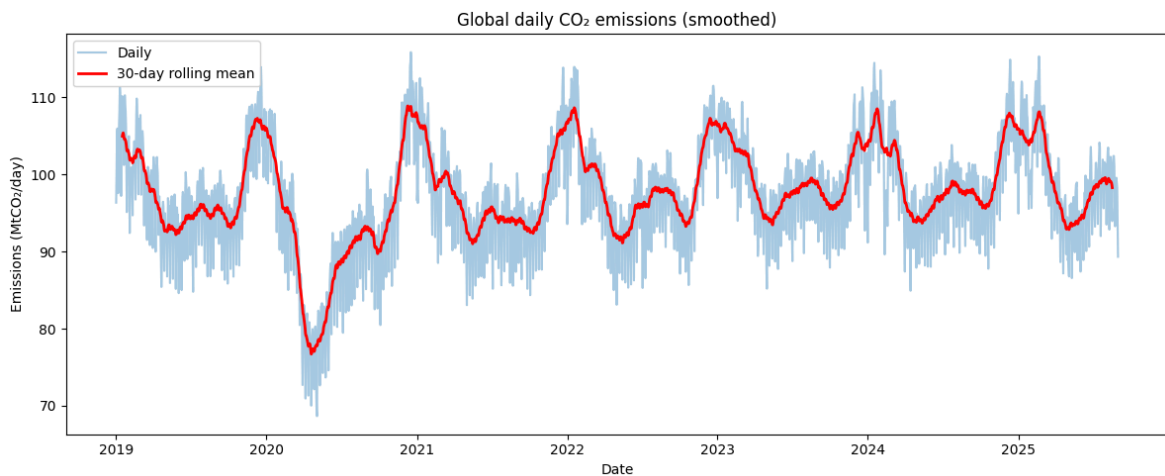
The plot shows the evolution of global daily CO₂ emissions (in MtCO₂/day) from 2019 to 2025. The time series exhibits a clear seasonal pattern, with regular annual peaks and troughs. A sharp drop is visible around early 2020, corresponding to the global lockdown period caused by the COVID-19 pandemic. After that, emissions gradually recovered and resumed their seasonal fluctuations. Overall, the global trend appears relatively stable over time, without a strong long-term increase or decrease.

To better analyze the temporal dynamics of global CO₂ emissions, it is useful to reduce the noise inherent in daily data. Daily fluctuations may reflect short-term events or reporting variability rather than genuine structural changes. Therefore, a 30-day rolling mean was applied to smooth the time series. This technique averages each value with its neighboring observations, helping to highlight medium-term trends and seasonal patterns while preserving the overall temporal resolution of the dataset.

```
In [ ]: # 30-day centered rolling mean to smooth daily fluctuations
world_daily['rolling_30d'] = world_daily['value'].rolling(window=30, center=True)

# Plot original and smoothed global CO2 emissions
plt.figure(figsize=(12, 5))
plt.plot(world_daily['date'], world_daily['value'], alpha=0.4, label='Daily')
plt.plot(world_daily['date'], world_daily['rolling_30d'], color='red', linewidth=2)

# Customize axes and legend for readability
plt.title('Global daily CO2 emissions (smoothed)')
plt.xlabel('Date')
plt.ylabel('Emissions (MtCO2/day)')
plt.legend()
plt.tight_layout()
plt.show()
```



The smoothed curve (in red) clearly reveals the underlying seasonal and structural patterns that were hidden in the raw daily data. We can observe a recurrent annual cycle, with regular peaks and troughs corresponding to global seasonal variations in economic and energy-related activity. The most notable drop occurs in early 2020, corresponding to the global reduction in activity during the COVID-19 lockdowns. After this period, emissions gradually recovered and resumed their cyclical pattern, stabilizing around similar seasonal amplitudes in subsequent years.

While the daily time series provides detailed information, it can be difficult to interpret broader trends due to high-frequency variability. To facilitate long-term analysis, the data were aggregated by month, computing the average daily CO₂ emissions for each month. This monthly aggregation highlights medium- and long-term dynamics, making it easier to identify structural changes, seasonal patterns, and periods of sustained increase or decrease in global emissions.

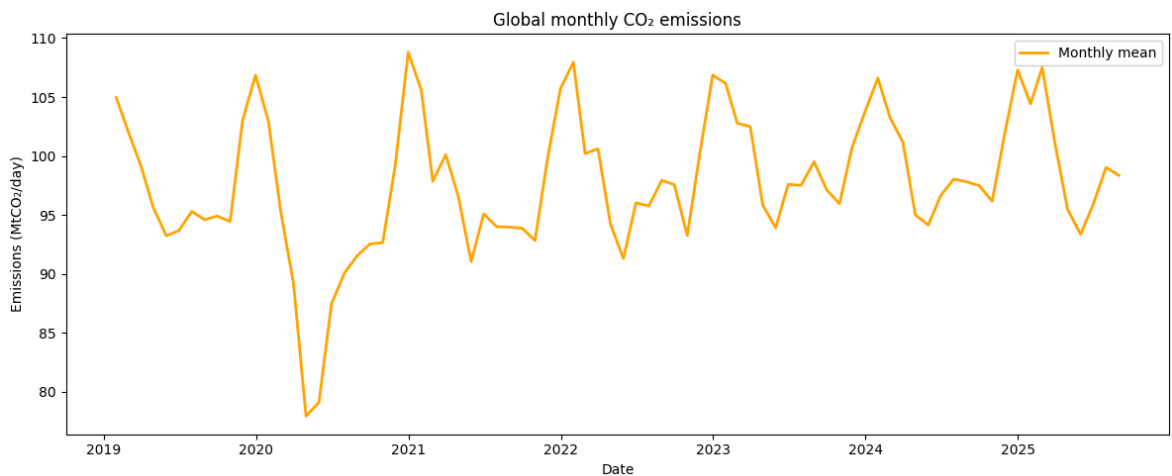
```
In [ ]: # Convert daily data to monthly frequency using the average value per month
world_monthly = world_daily.copy()
world_monthly['date'] = pd.to_datetime(world_monthly['date'])
world_monthly = world_monthly.resample('M', on='date')['value'].mean().reset_index()

# Plot monthly average CO2 emissions over time
plt.figure(figsize=(12, 5))
plt.plot(world_monthly['date'], world_monthly['value'], color='orange', linewidth=2)

# Add title, labels, and legend
plt.title('Global monthly CO2 emissions')
plt.xlabel('Date')
plt.ylabel('Emissions (MtCO2/day)')
plt.legend()
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-752759795.py:4: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.

```
world_monthly = world_monthly.resample('M', on='date')['value'].mean().reset_index()
```



The monthly mean series reveals clear recurring seasonal cycles, with regular peaks and troughs across years. A sharp decline is evident in early 2020, reflecting the drastic reduction in global emissions during the COVID-19 lockdowns. After that, emissions progressively recovered, stabilizing around pre-pandemic levels but maintaining the characteristic annual oscillations. The smoother, lower-frequency pattern confirms that global emissions fluctuate seasonally yet remain relatively stable in their long-term average over the period analyzed.

Before proceeding with anomaly detection, it is useful to summarize the main statistical characteristics of the monthly CO₂ emissions. Descriptive statistics provide an overview of the average level, dispersion, and range of variation in the data, allowing us to quantify the seasonal amplitude and identify potential outliers or periods of exceptional activity.

```
In [ ]: world_monthly['value'].describe()
```

```
Out[ ]:
```

	value
count	80.000000
mean	97.715447
std	5.793266
min	77.954224
25%	94.195971
50%	97.501863
75%	101.345642
max	108.801659

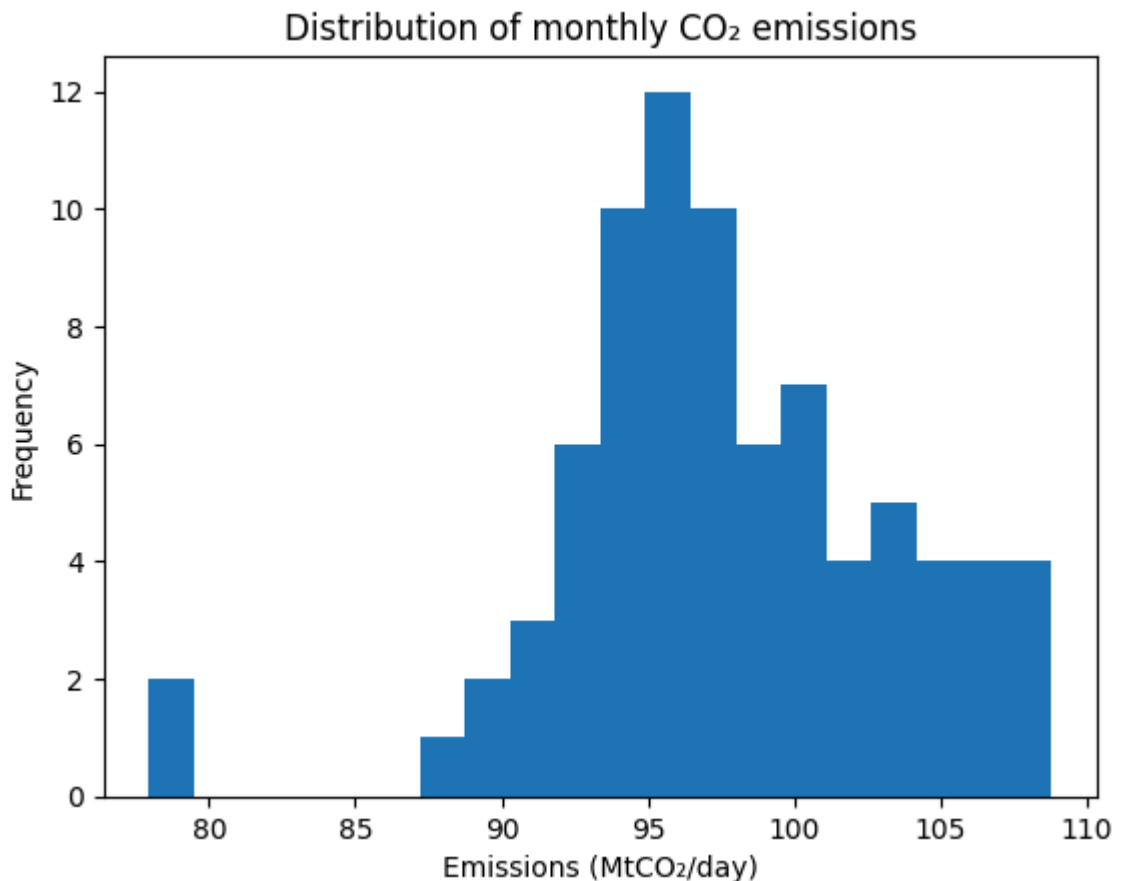
dtype: float64

The monthly series contains 80 observations, with an average emission level of about 97.7 MtCO₂/day. Values range between 77.9 MtCO₂/day (minimum) and 108.8 MtCO₂/day (maximum), showing a fairly wide variability consistent with seasonal fluctuations in global energy demand. The standard deviation of 5.8 MtCO₂/day indicates moderate dispersion around the mean, while the interquartile range (≈ 94.2 – 101.3 MtCO₂/day)

suggests that most months fall within this typical emission band. Overall, these statistics confirm the presence of regular oscillations rather than extreme instability, supporting the subsequent use of this series for anomaly detection based on deviations from expected seasonal behavior.

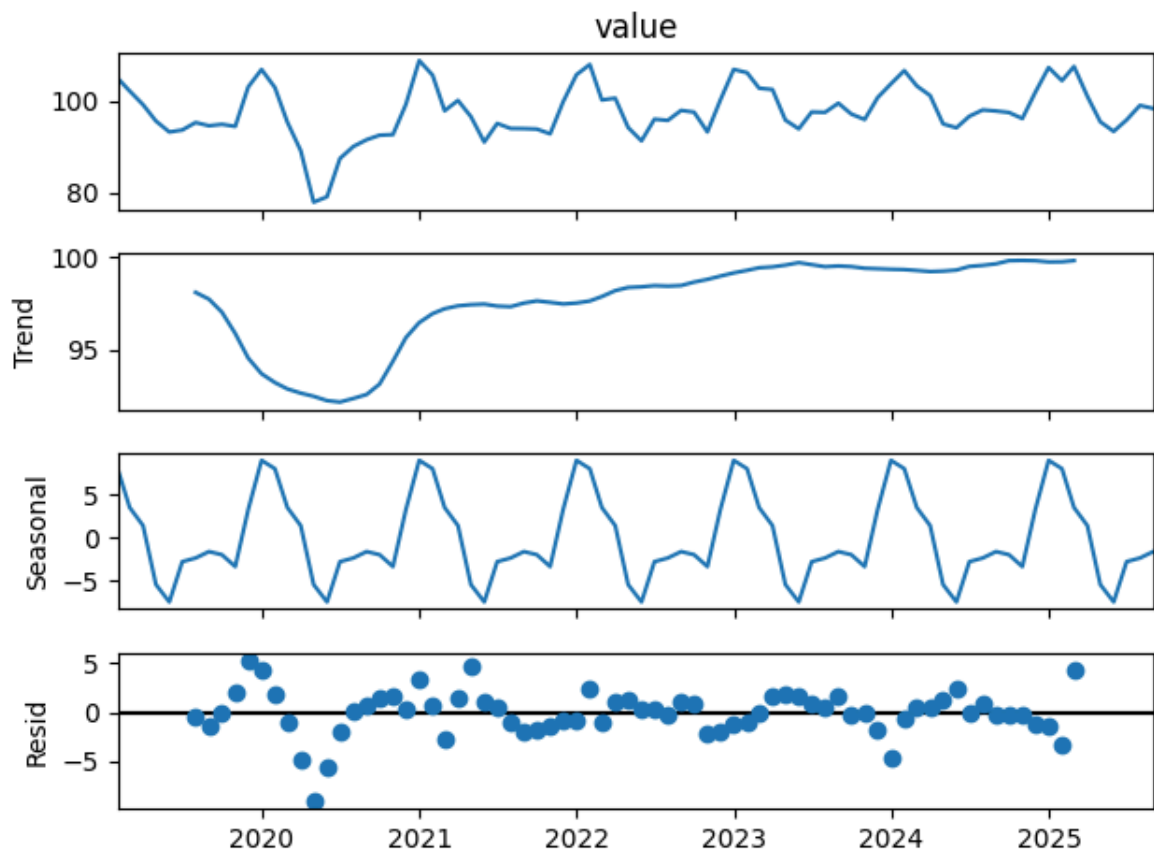
```
In [ ]: # Plot the distribution of monthly CO2 emissions
world_monthly['value'].plot(kind='hist', bins=20)

# Label axes and set chart title
plt.title('Distribution of monthly CO2 emissions')
plt.xlabel('Emissions (MtCO2/day)')
plt.show()
```



The histogram shows the distribution of monthly average CO₂ emissions over the analyzed period. The distribution is approximately unimodal and slightly right-skewed, with most observations concentrated between 90 and 105 MtCO₂/day. This indicates that global monthly emissions generally fluctuate within a relatively narrow range around their mean value (≈ 97.7 MtCO₂/day), confirming the presence of regular seasonal variation rather than extreme outliers. Only a few months record exceptionally low values (around 80 MtCO₂/day), which likely correspond to the global slowdown during the COVID-19 pandemic.

```
In [ ]: #Seasonal decomposition
decomp = seasonal_decompose(world_monthly.set_index('date')['value'], model='add')
decomp.plot()
plt.show()
```



The decomposition of the monthly CO₂ emission series highlights three key components.

- The trend component shows a marked decline during early 2020, coinciding with the global lockdowns, followed by a gradual recovery and stabilization near pre-pandemic levels.
- The seasonal component reveals a clear annual cycle, with consistent peaks and troughs that reflect recurring global patterns in energy consumption and industrial activity.
- The residual component oscillates around zero without large irregular shocks, suggesting that most of the variation in the series can be explained by trend and seasonality.

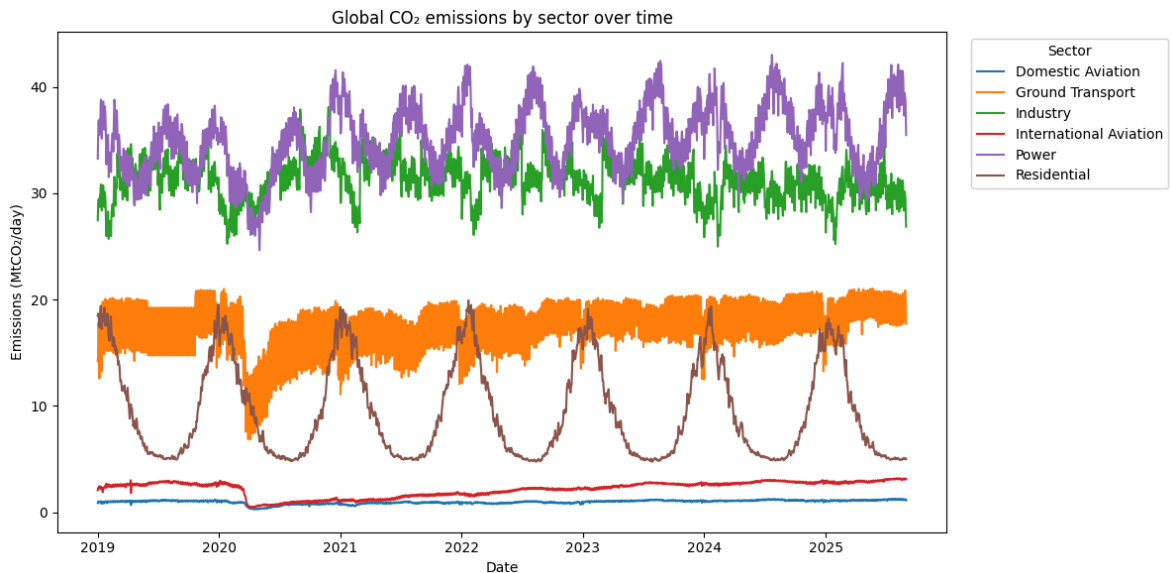
Overall, the decomposition confirms that global CO₂ emissions are characterized by strong seasonality and limited irregular noise, providing a robust foundation for subsequent anomaly detection based on deviations from the expected seasonal trend.

Global emission by sector

```
In [ ]: # Filter global data and compute daily emissions by sector
df_world = df[df['country'] == 'WORLD']
world_time_sector = (
    df_world.groupby(['date', 'sector'], as_index=False)['value'].sum()
)

# Plot CO2 emissions by sector over time
plt.figure(figsize=(12, 6))
sns.lineplot(data=world_time_sector, x='date', y='value', hue='sector', linewidth
```

```
# Title and axis names for clarity
plt.title('Global CO2 emissions by sector over time')
plt.xlabel('Date')
plt.ylabel('Emissions (MtCO2/day)')
plt.legend(title='Sector', bbox_to_anchor=(1.02, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



- Power and Industry are the dominant sources of emissions, showing clear seasonal oscillations likely linked to energy demand cycles.
- Ground Transport remains relatively stable but shows a visible drop during early 2020, corresponding to the COVID-19 lockdown period, followed by a recovery.
- Residential emissions display a strong winter seasonality, peaking each year due to heating demand.
- Domestic and International Aviation emissions are much lower in magnitude but exhibit a similar pandemic-related collapse in 2020, gradually returning to pre-crisis levels.

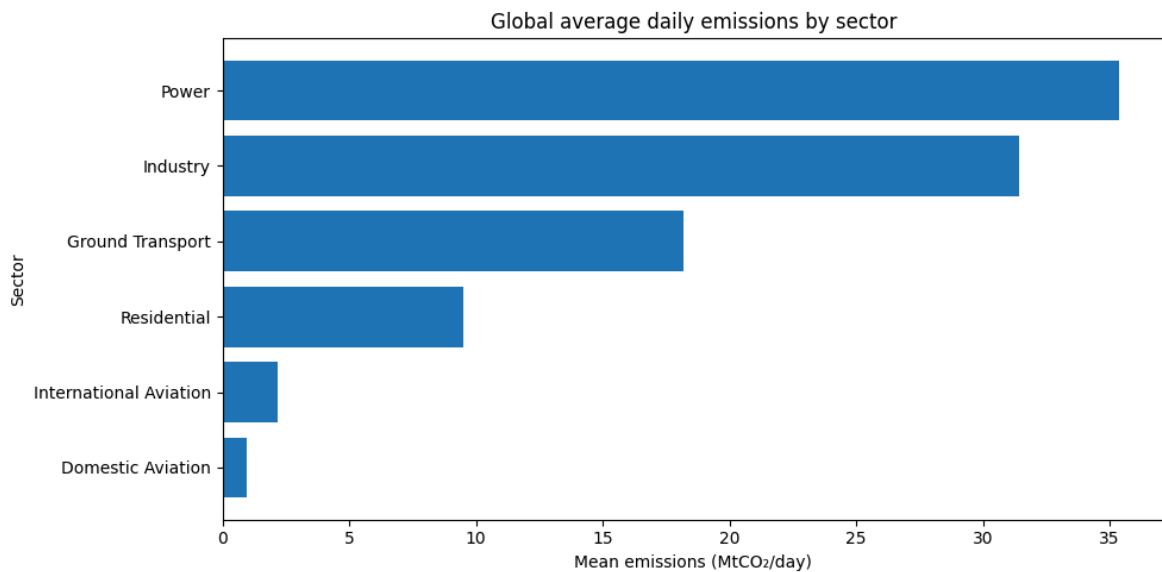
Overall, the figure highlights both the sectoral heterogeneity of CO₂ emissions and their sensitivity to global events and seasonal patterns.

```
In [ ]: # Mean daily emissions for each sector (global level)
world_sector_mean = (
    df_world.groupby('sector', as_index=False)['value']
        .mean()
        .sort_values('value', ascending=False)
)

# Plot average emissions by sector (horizontal bar chart)
plt.figure(figsize=(10, 5))
plt.barh(world_sector_mean['sector'], world_sector_mean['value'])
plt.gca().invert_yaxis() # Largest sector at the top

# Label axes and set chart title
```

```
plt.title('Global average daily emissions by sector')
plt.xlabel('Mean emissions (MtCO2/day)')
plt.ylabel('Sector')
plt.tight_layout()
plt.show()
```



Emissions of individual countries

```
In [ ]: def plot_country_emission_pie_with_labels_and_legend(df_filtered):
    """
    Plot a pie chart of total CO2 emissions by country (excluding 'World'),
    showing country names on the chart and full percentage breakdown in the legend
    """

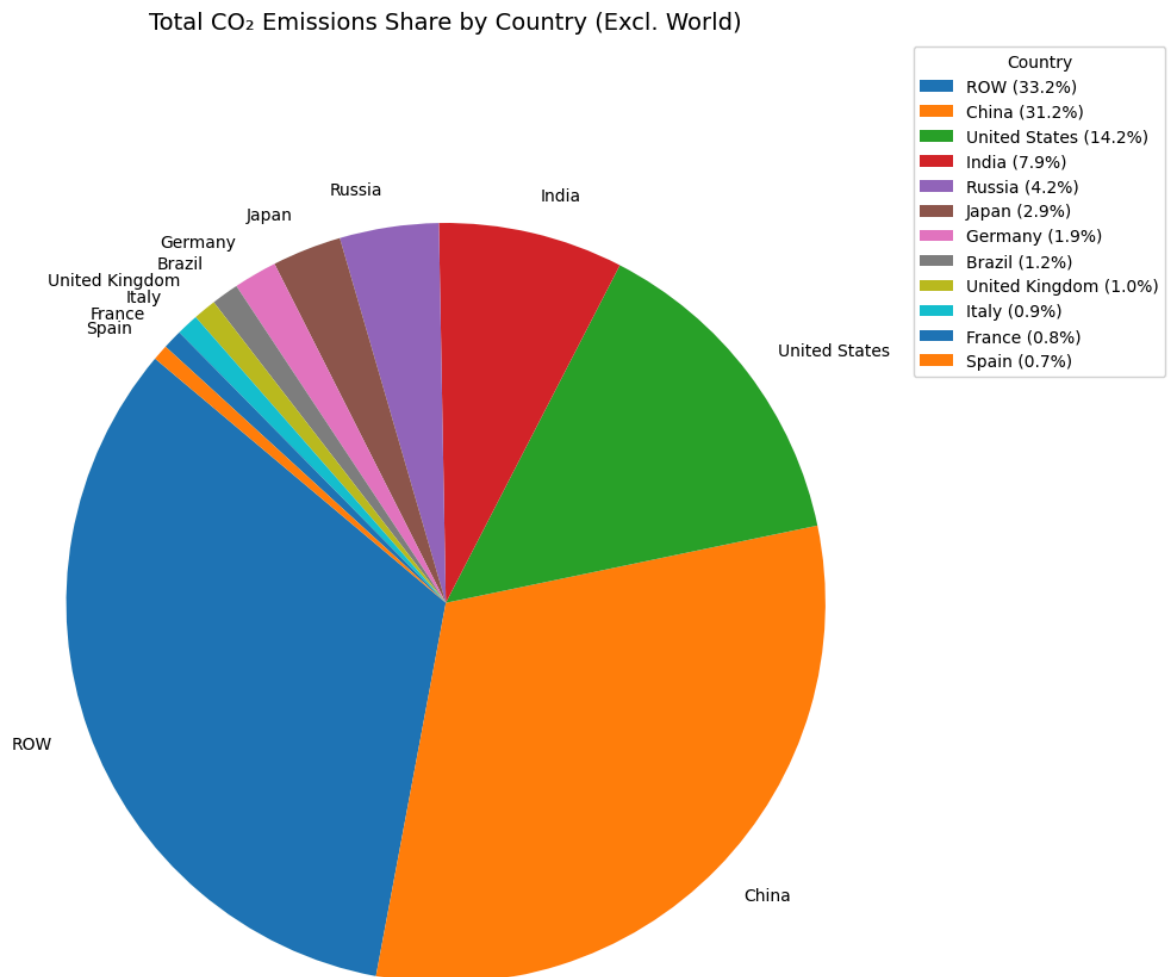
    # Total emissions per country
    total_emissions = df_filtered.groupby('country')['value'].sum().sort_values(
        total = total_emissions.sum()

    # Legend labels with percentage
    legend_labels = [f"{country} ({value / total * 100:.1f}%)" for country, value in total_emissions.items()]

    # Plot
    plt.figure(figsize=(10, 10)) # Reduced size
    wedges, texts = plt.pie(total_emissions, labels=total_emissions.index, startangle=90)

    # Add external legend
    plt.legend(wedges, legend_labels, title="Country", bbox_to_anchor=(1.05, 1),
    plt.title('Total CO2 Emissions Share by Country (Excl. World)', fontsize=14)
    plt.axis('equal')
    plt.tight_layout()
    plt.show()

df_excl_world_and_eu = df[(df['country'].str.upper() != 'WORLD') & (df['country'].str.upper() != 'EU')]
df_incl_eu_combined = df[(df['country'].str.upper() != 'WORLD') & (~df['country'].str.upper().isin(['EU']))]
plot_country_emission_pie_with_labels_and_legend(df_excl_world_and_eu)
#plot_country_emission_pie_with_labels_and_legend(df_incl_eu_combined)
```



The pie chart displays the share of total CO₂ emissions contributed by each country.

China (31.2%) and the Rest of the World (ROW, 33.2%) together account for nearly two-thirds of global emissions, emphasizing their dominant roles.

The United States contributes 14.2%, followed by India (7.9%), Russia (4.2%), and Japan (2.9%).

European countries such as Germany, the United Kingdom, Italy, France, and Spain each account for less than 2%, highlighting their comparatively smaller shares.

Overall, the figure shows a high concentration of emissions in a few major economies.

```
In [ ]: import matplotlib.dates as mdates

def plot_country_emissions_time_series(df, countries):
    """
    Plots the daily CO2 emissions over time for a list of specified countries.
    """
    plt.figure(figsize=(14, 8))
    for country in countries:
        df_country = df[df['country'] == country].groupby('date')['value'].sum()
```

```

plt.plot(df_country['date'], df_country['value'], label=country, linewidth

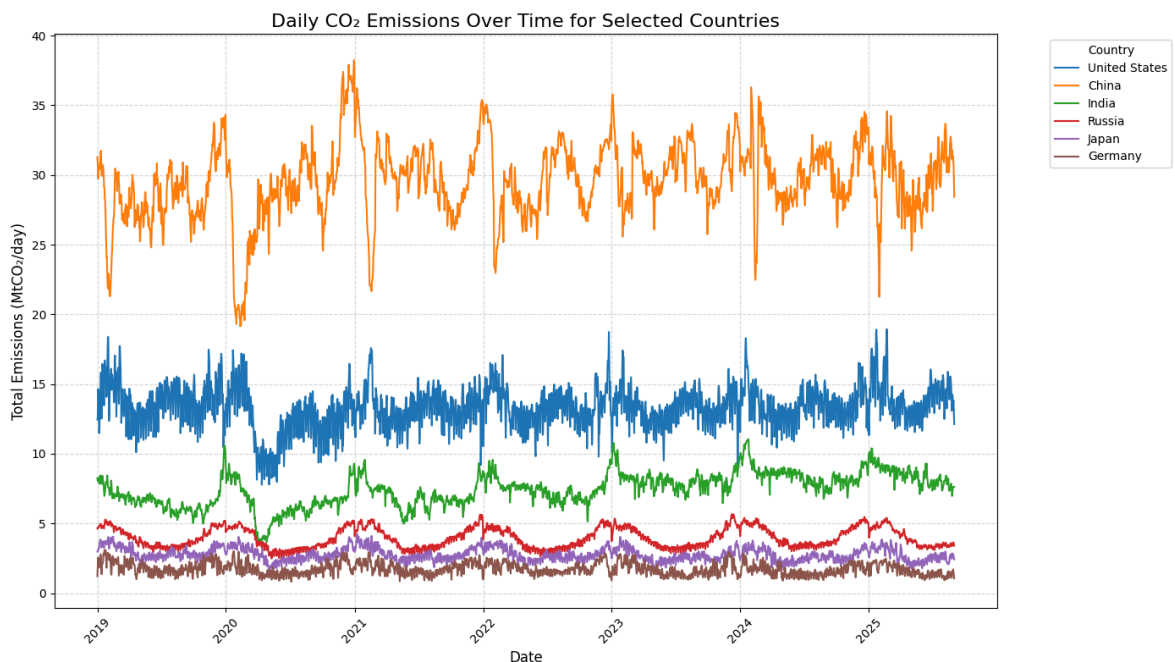
plt.title('Daily CO2 Emissions Over Time for Selected Countries', fontsize=1
plt.xlabel('Date', fontsize=12)
plt.ylabel('Total Emissions (MtCO2/day)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Country', bbox_to_anchor=(1.05, 1), loc='upper left')

# Format x-axis
plt.gca().xaxis.set_major_locator(mdates.YearLocator())
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# Select a few countries to plot
selected_countries = ['United States', 'China', 'India', 'Russia', 'Japan', 'Ger
plot_country_emissions_time_series(df, selected_countries)

```



The figure shows the daily evolution of CO₂ emissions (MtCO₂/day) from 2019 to 2025 for six major emitters: China, the United States, India, Russia, Japan, and Germany.

China consistently records the highest emission levels, displaying a regular seasonal pattern and a sharp decline during early 2020 due to COVID-19 lockdowns.

The United States follows as the second-largest emitter, with noticeable seasonal fluctuations and a similar pandemic-related dip.

India exhibits a gradual upward trend, reflecting its ongoing industrial and economic expansion.

Russia and Japan maintain moderate but steady emissions, while Germany contributes the least among the selected countries.

Overall, the plot highlights significant temporal variations and country-specific emission dynamics, emphasizing the global disparity in CO₂ output and the temporary impact of

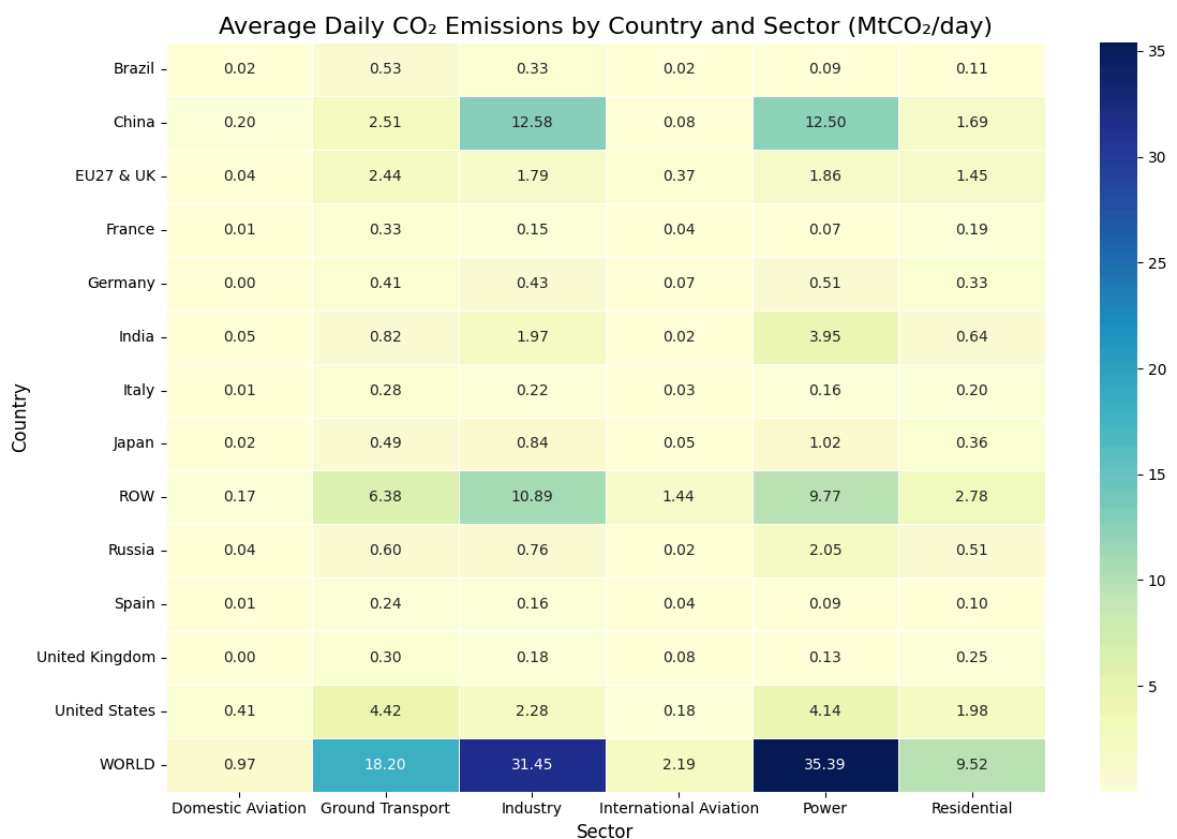
global disruptions like the pandemic.

```
In [ ]: # Group by country, sector, and date to compute mean daily emissions
df_grouped_heatmap = (
    df.groupby(['country', 'sector', 'date'])['value']
      .mean()
      .reset_index()
)

# Create pivot table: countries as rows, sectors as columns, mean emissions as values
# Averaged across all dates to show overall emission patterns
df_pivot_heatmap = df_grouped_heatmap.pivot_table(
    index='country',
    columns='sector',
    values='value',
    aggfunc='mean'
)

# Plot heatmap of average CO2 emissions by country and sector
plt.figure(figsize=(12, 8))
sns.heatmap(df_pivot_heatmap, annot=True, cmap='YlGnBu', fmt=".2f", linewidths=.5)

# Figure layout
plt.title('Average Daily CO2 Emissions by Country and Sector (MtCO2/day)', fontsize=12)
plt.xlabel('Sector', fontsize=12)
plt.ylabel('Country', fontsize=12)
plt.tight_layout()
plt.show()
```



This heatmap shows the average daily CO₂ emissions (in MtCO₂/day) for each country-sector pair. We can observe that:

- China is the largest emitter overall, particularly in the Industry and Power sectors, both exceeding 12 MtCO₂/day on average.
- The "ROW" (Rest of World) category also contributes significantly, especially in Industry and Power, indicating the impact of regions not individually listed.
- EU27 & UK and the United States follow as major contributors, with emissions spread across Ground Transport, Industry, and Power.
- Residential and Domestic Aviation sectors show relatively low emissions in all countries.
- The WORLD row aggregates the total emissions across all countries and highlights that Power and Industry are the dominant global sources of CO₂.

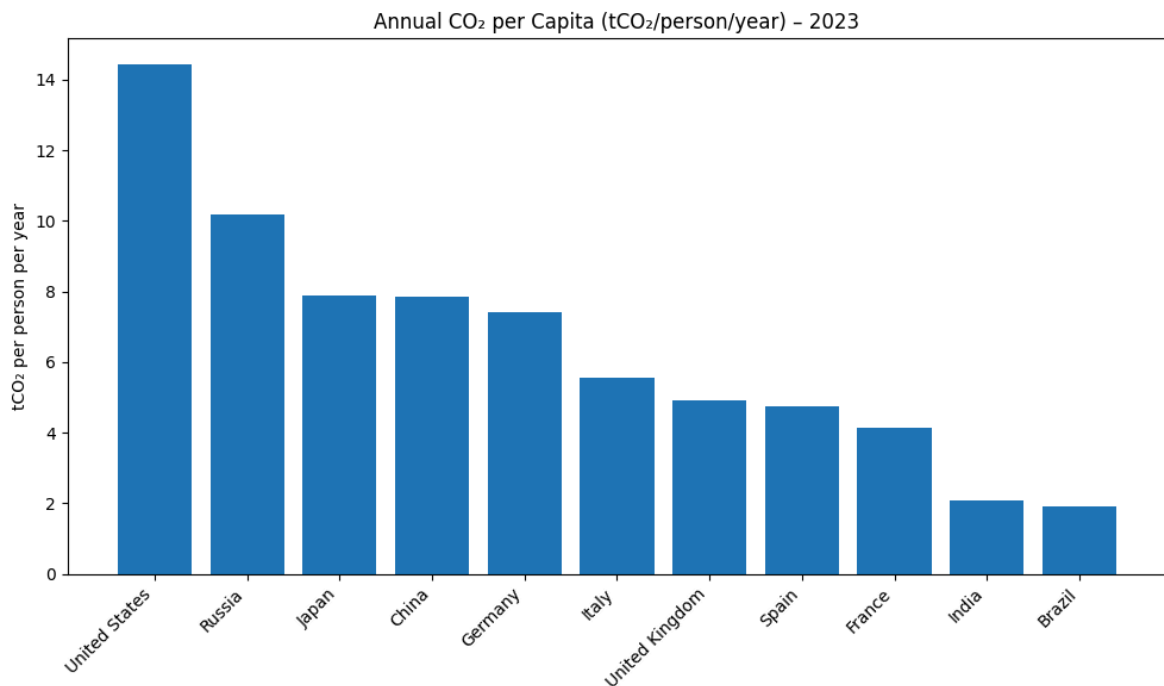
Overall, the figure clearly illustrates that energy generation and industrial activity are the main drivers of CO₂ emissions worldwide, with substantial variation among countries.

Normalized emissions

```
In [ ]: # Select data for a specific year and sort countries by per-capita emissions
year_to_plot = 2023
df_plot = (
    annual_df[annual_df["year"] == year_to_plot]
    .dropna(subset=["tCO2_per_capita_year"])
    .sort_values("tCO2_per_capita_year", ascending=False)
)

# Plot annual CO2 emissions per capita for each country
plt.figure(figsize=(10, 6))
plt.bar(df_plot["country_name"], df_plot["tCO2_per_capita_year"])

# Add title, labels, and rotated x-axis for readability
plt.title(f"Annual CO2 per Capita (tCO2/person/year) - {year_to_plot}")
plt.ylabel("tCO2 per person per year")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



The bar chart shows the annual CO₂ emissions per capita (in tonnes of CO₂ per person per year) across the selected countries for 2023.

The results highlight strong differences in individual carbon intensity:

- United States records the highest per-capita emissions, exceeding 14 tCO₂/person/year, mainly due to high energy consumption, transport reliance, and industrial output.
- Russia and Japan also show elevated values (around 10 tCO₂ and 8 tCO₂ per person), reflecting their industrial structure and energy mix.
- European countries such as Germany, Italy, Spain, and France cluster between 4 – 7 tCO₂/person/year, consistent with more efficient energy systems and higher shares of renewables.
- Emerging economies like India and Brazil have the lowest per-capita emissions (around 2 tCO₂/person/year), indicating lower consumption and industrialization levels per person, even though their total emissions are large due to population size.

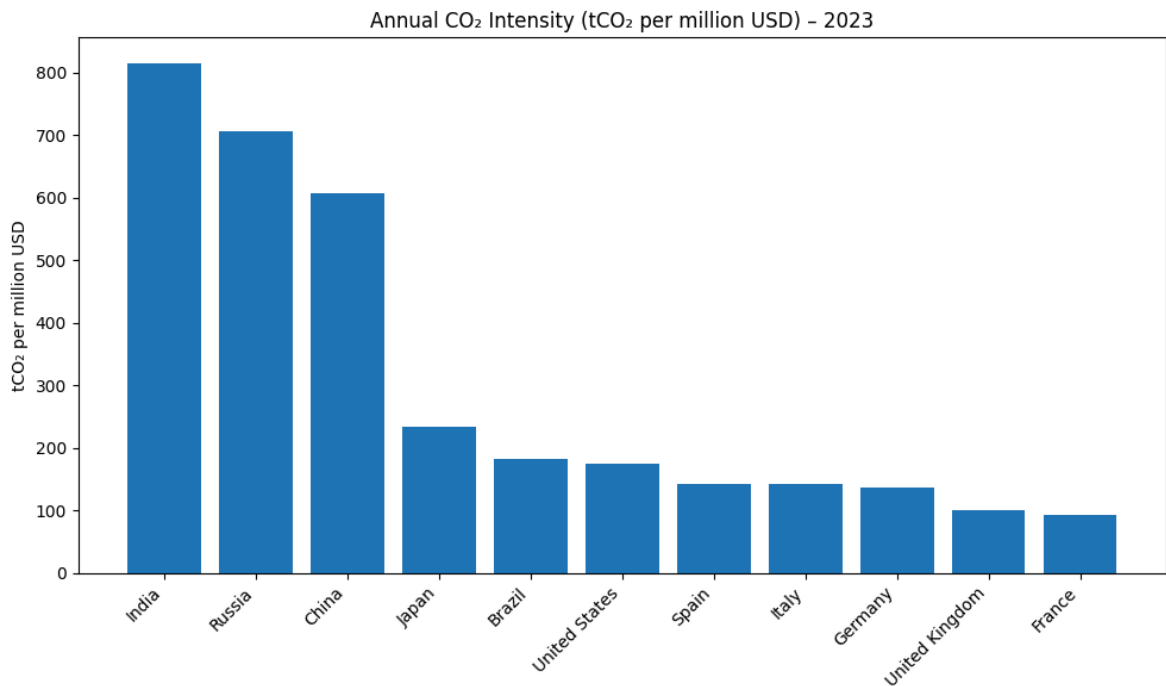
Overall, this comparison shows how population-normalized emissions reveal disparities in individual carbon footprints that are not visible when analyzing total emissions alone.

```
In [ ]: # Select data for a specific year and sort countries by CO2 intensity (per GDP u
year_to_plot = 2023
df_plot = (
    annual_df[annual_df["year"] == year_to_plot]
    .dropna(subset=["tCO2_per_million_USD_year"])
    .sort_values("tCO2_per_million_USD_year", ascending=False)
)

# Plot annual CO2 intensity (tons per million USD of GDP)
plt.figure(figsize=(10, 6))
plt.bar(df_plot["country_name"], df_plot["tCO2_per_million_USD_year"])

# Add title, labels, and rotate x-axis labels for clarity
plt.title(f"Annual CO2 Intensity (tCO2 per million USD) – {year_to_plot}")
```

```
plt.ylabel("tCO2 per million USD")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



This chart illustrates the carbon intensity of the economy, measured as tonnes of CO₂ emitted per million USD of GDP, for the year 2023.

The results reveal significant differences in the efficiency of economic production:

- India, Russia, and China show the highest carbon intensities, emitting more than 600–800 tCO₂ per million USD.
This suggests that their economies rely heavily on energy-intensive and carbon-based production processes, such as coal-fired electricity and heavy industry.
- Japan, Brazil, and the United States occupy the mid-range, reflecting mixed energy sources and moderate energy efficiency.
- European countries, especially France, United Kingdom, Germany, Italy, and Spain, display much lower intensities (below 150 tCO₂ per million USD), consistent with more efficient technologies, cleaner energy mixes, and stronger environmental regulations.

Overall, the comparison highlights how GDP-normalized emissions reveal differences in economic carbon efficiency.

While developing and industrializing nations generate more CO₂ per unit of output, advanced economies tend to produce more wealth with fewer emissions, reflecting progress toward decarbonization and sustainable growth.

4. Symbolic Aggregate approXimation (SAX)

To analyze temporal patterns of CO₂ emissions in a more compact and comparable way, we applied the Symbolic Aggregate approXimation (SAX) method. SAX converts each

country's normalized time series into a symbolic representation by:

- Normalizing the series (zero mean, unit variance);
- Smoothing the signal via a rolling mean (Piecewise Aggregate Approximation, PAA);
- Discretizing the smoothed values into a small set of bins based on normal distribution breakpoints;
- Mapping each bin to a symbol (from 'a' to 'e').

The SAX parameters were selected to balance temporal resolution and interpretability. A window size of 30 days was chosen because the dataset contains daily CO₂ emission values, and this interval roughly corresponds to one month of observations. It provides sufficient smoothing to remove short-term noise while preserving meaningful monthly and seasonal variations in emissions.

The number of bins was set to 5, which produces a symbolic alphabet of five discrete levels (a–e). This choice offers a good trade-off between detail and generalization: using too few bins would oversimplify the dynamics, while too many bins would create overly sparse and hard-to-interpret patterns.

Preliminary visual inspections confirmed that these parameters generate sequences that are both informative and stable, allowing for a clear qualitative comparison across countries and a reliable anomaly detection based on rare symbolic patterns.

This symbolic representation allows qualitative comparison of emission patterns across countries and facilitates tasks such as pattern discovery, clustering, and anomaly detection, while significantly reducing dimensionality.

```
In [ ]: # Parameters
window_size = 30
n_bins = 5
breakpoints = norm.ppf(np.linspace(0, 1, n_bins + 1)[1:-1])
symbol_map = {0: 'a', 1: 'b', 2: 'c', 3: 'd', 4: 'e'}

# Ensure 'date' is in datetime format
df['date'] = pd.to_datetime(df['date'])

# Aggregate CO2 values by country and date
df_country_grouped = df.groupby(['country', 'date'], as_index=False)['value'].sum()

# Dictionary to store SAX sequences
sax_by_country = {}

# Generate SAX sequence for each country
for country in df_country_grouped['country'].unique():
    df_country = df_country_grouped[df_country_grouped['country'] == country].copy()
    df_country = df_country.set_index('date').sort_index()

    # Normalize time series
    scaler = StandardScaler()
    df_country['value_norm'] = scaler.fit_transform(df_country[['value']])
```

```
# Piecewise Aggregate Approximation (PAA)
paa_values = df_country['value_norm'].rolling(window=window_size).mean().dro

# SAX discretization
symbols = np.digitize(paa_values, breakpoints)
sax_sequence = ''.join([symbol_map[i] for i in symbols])

# Store result
sax_by_country[country] = sax_sequence

# Show first 3 SAX sequences
list(sax_by_country.items())[:3]
```

[illegible]

```

ddddddeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeddddddccccbbbbbbbbbbbbbb
bbbbbbcccccccccccccccccccccccccccccccccccccccccccccccccccccccc
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbcccccccccccccccccccc
ccdddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccc'),
('EU27 & UK',
 'eeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeddeeeeddddddcccccccccccccccc
eeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
eeeeeeeeeeeeeeeeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccc
eeeeeeeeeeeeeeeeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccc
dddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
abbbbaabbbbaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbcccccccccccc
cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
dddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
dddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
dddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbaabaaaaaaaaaaaaaa
aaaaabbbbbbbbbbbbbbbcccccccccccccccccccccccccccccccccccccccccccccccccccccccc
ddddddeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeeddddddcccccccccccccccccccccccccccccccc
eeeeeeeeeeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
eeeeeeeeeeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
eeeeeeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
bbbbbbbbbbbbbbbbbaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbaabaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
bbbbbbbbbcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
cccddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbaaaabaaabbbbaabbbba
bbbbbbabbbbaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaababbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbcbbbbcccccccccccccccc
dddddcccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
eeeeedddddeeeeeeeeddddddcccccccccccccccccccccccccccccccccccccccccccccccccccc
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbb
bbbbbbbaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa')]
```

The heatmap below shows the evolution of these symbolic sequences over time, where colors represent the relative emission levels within each country's historical context.

```

In [ ]: # Select countries to visualize
selected_countries = ['United States', 'China', 'Russia', 'India', 'Spain', 'Ita

# Filter only the selected countries
filtered_sax = {country: seq for country, seq in sax_by_country.items() if count

# Ensure equal sequence length (truncate to the minimum)
min_len = min(len(seq) for seq in filtered_sax.values())
trimmed_sax = {country: list(seq[:min_len]) for country, seq in filtered_sax.ite

# Convert to DataFrame (countries as rows)
sax_df = pd.DataFrame(trimmed_sax).T

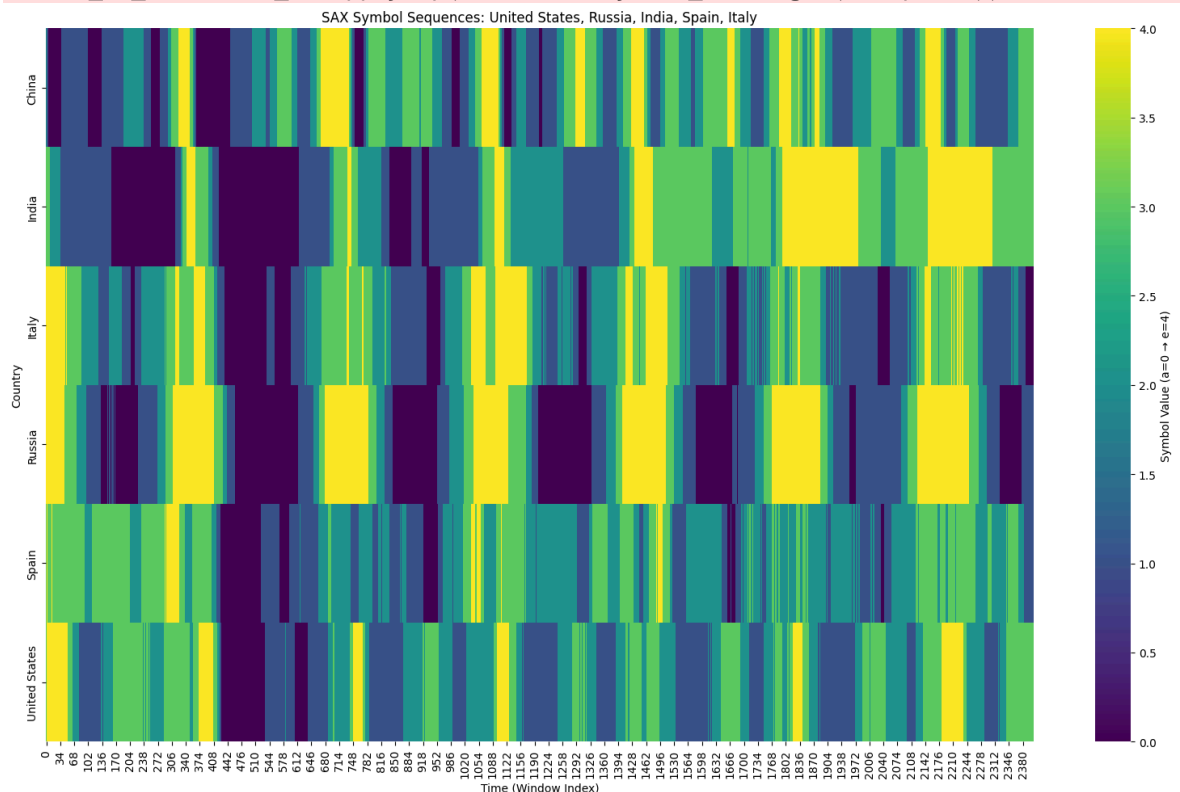
# Map symbols ('a'-'e') to numeric values (0-4) for visualization
symbol_order = {'a': 0, 'b': 1, 'c': 2, 'd': 3, 'e': 4}
sax_df_num = sax_df.applymap(lambda x: symbol_order.get(x, np.nan))

# Plot heatmap
plt.figure(figsize=(16, 10))
```

```
sns.heatmap(
    sax_df_num,
    cmap='viridis',
    cbar_kws={'label': 'Symbol Value (a=0 → e=4)'}
)
plt.title("SAX Symbol Sequences: United States, Russia, India, Spain, Italy")
plt.xlabel("Time (Window Index)")
plt.ylabel("Country")
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-1459859694.py:16: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.

```
sax_df_num = sax_df.applymap(lambda x: symbol_order.get(x, np.nan))
```



The heatmap highlights distinct temporal patterns among countries. For instance, China and India display longer periods of high symbolic values (yellow tones), reflecting generally increasing or sustained emission levels. In contrast, countries such as Italy and Spain exhibit more frequent transitions between low and high symbols, indicating greater variability and possible structural changes. Overall, the visualization suggests heterogeneous dynamics across countries, with certain global phases where several nations experience simultaneous changes in emission patterns.

5. Anomaly detection

In this section we apply an anomaly detection technique based on the Symbolic Aggregate approXimation (SAX) representation. After converting each country's CO₂ emission time series into symbolic sequences (with symbols from a to e), we search for unusual patterns that rarely occur in the historical data.

We split each symbolic sequence into small overlapping patterns of length n (called n -grams).

We estimate the frequency of each pattern across all countries and compute its probability with Laplace smoothing.

Each window of the series receives an anomaly score equal to the negative logarithm of its pattern probability ($-\log p$).

The higher the score, the more unusual the local behavior of emissions.

We then identify and visualize the top- k anomalous windows (the rarest symbolic patterns) for both the global emissions (WORLD) and a selected country (Italy). Red shaded areas in the plots correspond to the time intervals flagged as potentially anomalous.

```
In [ ]: from collections import Counter

def ngrams(seq, n=3):
    return [seq[i:i+n] for i in range(len(seq)-n+1)]

def fit_ngram_model(seqs_dict, n=3, alpha=1.0):
    counts = Counter()
    for s in seqs_dict.values():
        counts.update(ngrams(s, n))
    N = sum(counts.values())
    V = max(len(counts), 1)
    probs = {g: (c + alpha) / (N + alpha * V) for g, c in counts.items()}
    default_p = alpha / (N + alpha * V)
    return probs, default_p

def dates_for_country(df_country_grouped, country):
    return (df_country_grouped[df_country_grouped['country'] == country]
            .sort_values('date')['date'].reset_index(drop=True))

In [ ]: def detect_anomalies_for_country(country, sax_by_country, df_country_grouped,
                                         probs, default_p, window_size=30, n=3, top_k=5)

    # sequence SAX
    if country not in sax_by_country or len(sax_by_country[country]) < n:
        return pd.DataFrame(columns=['country', 'pattern', 'score', 'start_date', 'end_date'])
    seq = sax_by_country[country]

    # scores = -Log p(ngram)
    gs = ngrams(seq, n)
    p = np.array([probs.get(g, default_p) for g in gs])
    scores = -np.log(p)

    # top-k indexes
    idx = np.argsort(scores)[-top_k:][::-1]

    # mapping symbols
    dates = dates_for_country(df_country_grouped, country)
    symbol_dates = dates[window_size-1 : window_size-1 + len(seq)]

    rows = []
    for i in idx:
```

```

start_date = symbol_dates.iloc[i]
end_date = symbol_dates.iloc[min(i + n - 1, len(symbol_dates)-1)]
rows.append({
    'country': country,
    'pattern': gs[i],
    'score': float(scores[i]),
    'start_date': start_date,
    'end_date': end_date
})
return pd.DataFrame(rows).sort_values('score', ascending=False).reset_index(

```

```

In [ ]: def plot_country_with_anomalies(country, df_country_grouped, df_anoms):
    dfc = (df_country_grouped[df_country_grouped['country'] == country]
           .sort_values('date'))
    plt.figure(figsize=(12,5))
    plt.plot(dfc['date'], dfc['value'], label=f'{country} CO2 series', color='b')

    for _, r in df_anoms.iterrows():
        plt.axvspan(r['start_date'], r['end_date'], color='red', alpha=0.3)

    plt.title(f"SAX-based Anomaly Detection: {country}")
    plt.xlabel("Date"); plt.ylabel("CO2 emissions (MtCO2/day)")
    plt.legend(); plt.tight_layout(); plt.show()

```

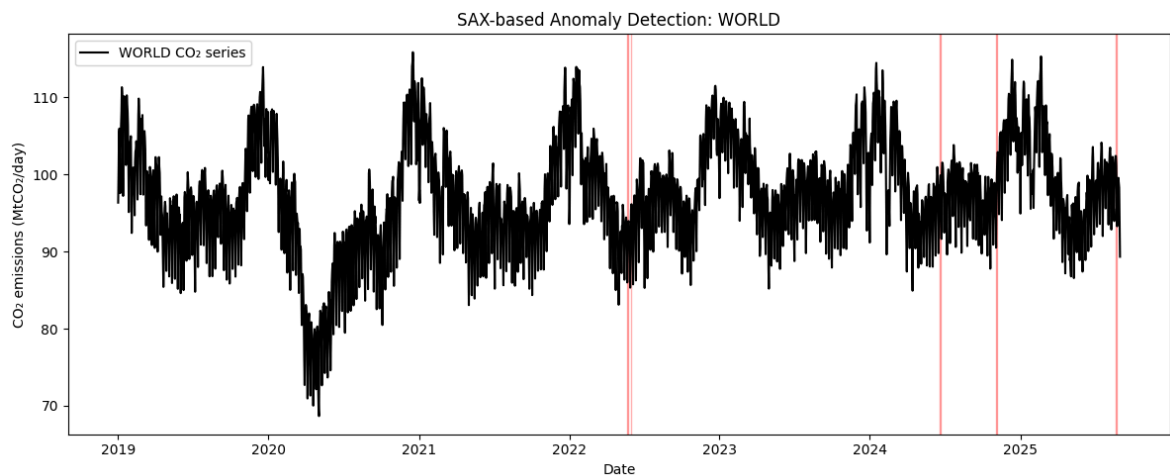
```

In [ ]: # Probability model
n = 3
probs, default_p = fit_ngram_model(sax_by_country, n=n, alpha=1.0)

#WORLD
df_anoms_world = detect_anomalies_for_country(
    "WORLD", sax_by_country, df_country_grouped,
    probs, default_p, window_size=window_size, n=n, top_k=5
)
display(df_anoms_world)
plot_country_with_anomalies("WORLD", df_country_grouped, df_anoms_world)

```

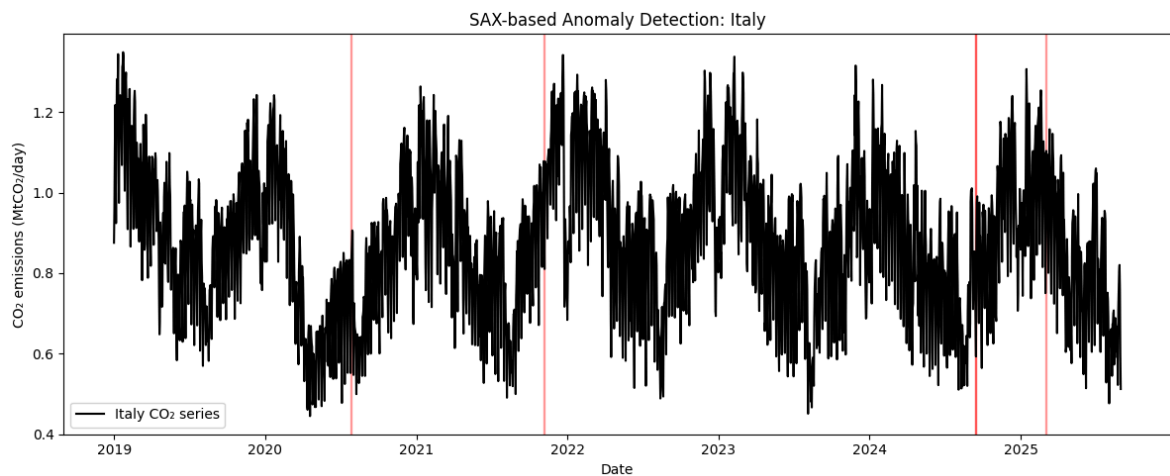
	country	pattern	score	start_date	end_date
0	WORLD	cdc	7.480369	2025-08-21	2025-08-23
1	WORLD	bcb	7.092603	2024-06-20	2024-06-22
2	WORLD	cbc	6.928300	2024-11-02	2024-11-04
3	WORLD	aab	5.829688	2022-05-22	2022-05-24
4	WORLD	aab	5.829688	2022-05-30	2022-06-01



The SAX-based anomaly detection highlights a few short time windows where the symbolic patterns are statistically rare in the global CO₂ emission series. The detected anomalies (for example around May–June 2022, mid-2024, and August 2025) correspond to local fluctuations that deviate from the dominant seasonal trend. These periods likely reflect short-term irregularities in global activity or measurement noise rather than structural breaks, since the overall yearly cycles remain consistent. Nevertheless, the method effectively identifies the most unusual symbolic behaviors in the series, showing its usefulness for locating potential anomalies in a simple and interpretable way.

```
In [ ]: #ITALY
df_anoms_italy = detect_anomalies_for_country(
    "Italy", sax_by_country, df_country_grouped,
    probs, default_p, window_size=window_size, n=n, top_k=5
)
display(df_anoms_italy)
plot_country_with_anomalies("Italy", df_country_grouped, df_anoms_italy)
```

	country	pattern	score	start_date	end_date
0	Italy	ede	8.345366	2025-03-02	2025-03-04
1	Italy	aba	7.939901	2024-09-13	2024-09-15
2	Italy	cdc	7.480369	2021-11-05	2021-11-07
3	Italy	bab	7.289314	2024-09-14	2024-09-16
4	Italy	bab	7.289314	2020-07-25	2020-07-27



6. Structural Break Detection

CUSUM test

The CUSUM (Cumulative Sum of Residuals) test was applied to detect structural breaks in the global CO₂ emission trend. The method fits a simple linear model to the time series and evaluates whether the cumulative sum of residuals deviates significantly from zero. The chart below shows the evolution of the CUSUM statistic with 95% confidence bands (grey dashed lines). A large deviation crossing these limits indicates a significant structural change.

In our case, a major breakpoint was detected around early 2020, which is consistent with the sharp global decline in emissions during the COVID-19 lockdowns. After that point, the CUSUM statistic stabilizes again, suggesting a new emission regime.

```
In [ ]: from statsmodels.stats.diagnostic import breaks_cusumolsresid
        from statsmodels.api import add_constant, OLS

        # Prepare WORLD data
        df_world = df[df['country'] == 'WORLD'].copy()
        df_world = df_world.groupby('date')['value'].sum().reset_index()
        df_world['date'] = pd.to_datetime(df_world['date'])
        df_world = df_world.sort_values('date').reset_index(drop=True)
        df_world['time_index'] = np.arange(len(df_world))

        # Fit linear trend model
        X_world = add_constant(df_world['time_index'])
        y_world = df_world['value']
        model_world = OLS(y_world, X_world).fit()

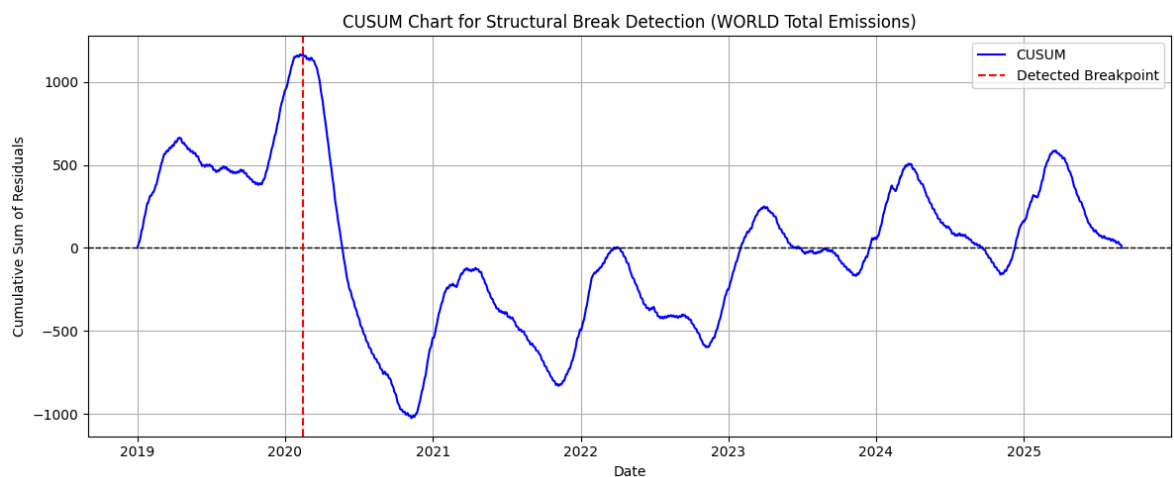
        # Run CUSUM test
        cusum_stat_world, p_value_world, crit_vals_world = breaks_cusumolsresid(model_world)

        # Compute cumulative residuals for visualization
        residuals_world = model_world.resid
        cusum_world = (residuals_world - residuals_world.mean()).cumsum()

        # Identify possible structural break date
        max_dev_index_world = cusum_world.abs().idxmax()
        breakpoint_date_world = df_world.loc[max_dev_index_world, 'date']
```

```
# Plot CUSUM chart with breakpoint Line
plt.figure(figsize=(12, 5))
plt.plot(df_world['date'], cusum_world, label='CUSUM', color='blue')
plt.axhline(0, color='black', linestyle='--', linewidth=1)
plt.axvline(breakpoint_date_world, color='red', linestyle='--', linewidth=1.5, label='Detected Breakpoint')
plt.title('CUSUM Chart for Structural Break Detection (WORLD Total Emissions)')
plt.xlabel('Date')
plt.ylabel('Cumulative Sum of Residuals')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Display results
print(f"Possible Structural Break Date: {breakpoint_date_world.date()}")
print(f"CUSUM Statistic: {cusum_stat_world:.4f}")
print(f"p-value: {p_value_world:.4f}")
print(f"Critical Values: {crit_vals_world}")
```



```
Possible Structural Break Date: 2020-02-14
CUSUM Statistic: 3.4495
p-value: 0.0000
Critical Values: [(1, 1.63), (5, 1.36), (10, 1.22)]
```

The CUSUM test clearly indicates a significant structural break in the global CO₂ emission trend. The statistic value (3.45) largely exceeds the 5% critical value (1.36), and the associated p-value (0.0000) confirms that the null hypothesis of parameter stability can be rejected. The detected breakpoint is dated February 14, 2020, which coincides with the beginning of the sharp global decline in emissions caused by the COVID-19 lockdowns. This result confirms the presence of a major structural change in the emission dynamics at the start of 2020, followed by a new regime of more stable fluctuations in subsequent years.

7. Clustering Analysis

Two complementary clustering approaches are explored. The first grouped countries according to the shape of their emission time series, capturing similarities in temporal

evolution regardless of scale. The second focuses on absolute levels and long-term trends, highlighting structural differences in emission magnitude.

7.1 Clustering analysis based on temporal evolution of emissions

This section aims to group countries according to the *temporal evolution* of their daily CO₂ emissions rather than their absolute emission levels.

To achieve this, each country's emission time series was first aggregated at the daily level and then standardized (z-score) across time, ensuring that all series share a common scale with mean zero and unit variance.

This transformation removes differences in emission magnitude and highlights relative temporal fluctuations — thus allowing the analysis to focus on the *shape* of emission dynamics over time.

A Principal Component Analysis (PCA) was subsequently applied to reduce the dimensionality of the standardized dataset while preserving 95% of its variance. The reduced representation was then used as input for the K-Means algorithm to identify groups of countries exhibiting similar emission trajectories. The optimal number of clusters was determined by combining the Elbow and Silhouette methods, balancing compactness, separation, and interpretability.

This temporal-pattern clustering provides insights into how different countries responded to global shocks (e.g., the 2020 pandemic) and how their emission behaviors evolved over recent years, revealing similarities and divergences in their environmental and economic dynamics.

```
In [ ]: df_work = df.copy()
df_work['date'] = pd.to_datetime(df_work['date'], dayfirst=True, errors='coerce')
df_work = df_work.sort_values('date')

# Remove global/aggregate rows
df_noworld = df_work.loc[df_work['country'].str.upper().ne('WORLD')].copy()
df_noworld = df_noworld[df_noworld['country'].str.upper() != 'ROW']

# Daily totals by country (summing across sectors)
country_daily = (
    df_noworld.groupby(['country', 'date'], as_index=False)['value'].sum()
)

# Build a complete, aligned date index to ensure comparable time axes
all_dates = pd.date_range(
    start=country_daily['date'].min(),
    end=country_daily['date'].max(),
    freq='D'
)
```

```
In [ ]: # Pivot to wide: rows=countries, cols=dates; reindex to full calendar
df_pivot = (country_daily
    .pivot(index='country', columns='date', values='value')
    .reindex(columns=all_dates)
    .sort_index(axis=1)           # ordina colonne per data
    .astype(float))              # tipo numerico
```

```
print("Shape matrix (countries x time):", df_pivot.shape)
```

Shape matrix (countries x time): (12, 2435)

```
In [ ]: #Data quality checks
n_missing = int(df_pivot.isna().sum().sum())
per_country_missing = df_pivot.isna().mean(axis=1).sort_values(ascending=False)

print(f"Missing totali nella matrice: {n_missing}")
print("Top paesi per percentuale di missing:")
print(per_country_missing.head(10))
```

Missing totali nella matrice: 0

Top paesi per percentuale di missing:

country

Brazil	0.0
China	0.0
EU27 & UK	0.0
France	0.0
Germany	0.0
India	0.0
Italy	0.0
Japan	0.0
Russia	0.0
Spain	0.0

dtype: float64

Standardization of country-level time series

```
In [ ]: X = df_pivot.copy()
mu = X.mean(axis=1)
sigma = X.std(axis=1, ddof=0).replace(0, np.nan)
X_scaled = X.sub(mu, axis=0).div(sigma, axis=0)
X_scaled = X_scaled.dropna(how='all')
print("Data standardized by country (mean=0, std=1). Shape:", X_scaled.shape)
```

Data standardized by country (mean=0, std=1). Shape: (12, 2435)

Dimensionality reduction + model selection for K-Means

```
In [ ]: #Dimensionality reduction (PCA space for clustering)
pca = PCA(n_components=0.95, svd_solver='full')
X_pca = pca.fit_transform(X_scaled)

print("Explained variance ratio:", pca.explained_variance_ratio_.sum())

# Metrics across candidate k
inertia = []
sil_scores = []
K_range = range(2, 6) # max 6 clusters for 13 countries

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = kmeans.fit_predict(X_pca)
    inertia.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(X_pca, labels))

# Visualization of Elbow & Silhouette
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
```

```

chosen_k = 3 # o 4
ax[0].axvline(chosen_k, ls='--', alpha=0.7)
ax[1].axvline(chosen_k, ls='--', alpha=0.7)

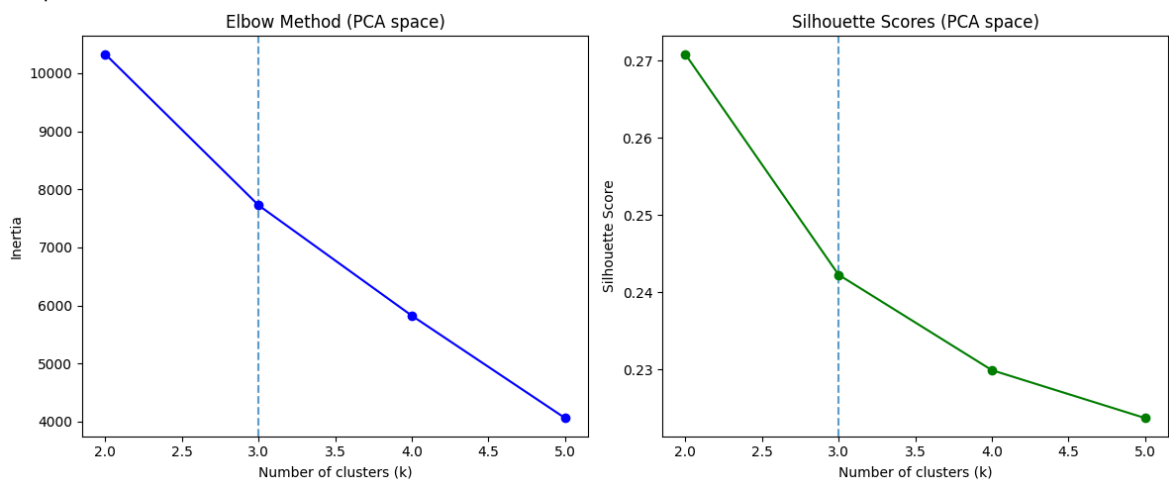
ax[0].plot(K_range, inertia, 'o-', color='blue')
ax[0].set_title('Elbow Method (PCA space)')
ax[0].set_xlabel('Number of clusters (k)')
ax[0].set_ylabel('Inertia')

ax[1].plot(K_range, sil_scores, 'o-', color='green')
ax[1].set_title('Silhouette Scores (PCA space)')
ax[1].set_xlabel('Number of clusters (k)')
ax[1].set_ylabel('Silhouette Score')

plt.tight_layout()
plt.show()

```

Explained variance ratio: 0.9585727004979718



The two plots illustrate the evaluation of potential cluster counts (k) using the Elbow and Silhouette methods after applying PCA (retaining 95% of the total variance).

In the Elbow plot, inertia decreases sharply up to $k = 3$, after which the reduction becomes more gradual, suggesting diminishing returns in cluster compactness. This inflection point indicates that three clusters provide an adequate trade-off between model simplicity and within-cluster homogeneity.

In the Silhouette plot, the silhouette coefficient reaches its maximum for $k = 2$, but the value for $k = 3$ remains relatively high and then steadily declines for larger k . This pattern implies that while two clusters yield the highest separation, a three-cluster solution preserves good structure while offering greater interpretability.

Overall, combining both criteria suggests that $k = 3$ is the most appropriate choice: it balances compactness, separation, and interpretability in the PCA-reduced space.

Fit final KMeans model (choose k manually after plots)

```
In [ ]: X_pca_km = X_pca
```

```
# K-Means on PCA-reduced data
```

```

optimal_k = 3 # o 3, in base alla scelta finale
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=50)
labels = kmeans.fit_predict(X_pca_km)

# Store Labels separately (preserve df_pivot numeric structure)
clusters_df = pd.DataFrame({'country': X_scaled.index, 'cluster': labels}).set_i

# Diagnostics: separation and cluster sizes
sil = silhouette_score(X_pca_km, labels)
sizes = clusters_df['cluster'].value_counts().sort_index()
print(f"Silhouette (PCA space, k={optimal_k}): {sil:.3f}")
print("Cluster sizes:\n", sizes)

# 2D visualization on the first two principal components (same PCA space)
X_pca2 = X_pca_km[:, :2] # PC1, PC2

plt.figure(figsize=(9, 7))
sns.scatterplot(x=X_pca2[:, 0], y=X_pca2[:, 1], hue=labels, palette='Set2', s=11)
for i, country in enumerate(X_scaled.index):
    plt.text(X_pca2[i, 0] + 0.02, X_pca2[i, 1] + 0.02, country, fontsize=8, alph

plt.title(f'Countries clustered in PCA space (k={optimal_k})')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend(title='Cluster', loc='best', frameon=True)
plt.tight_layout()
plt.show()

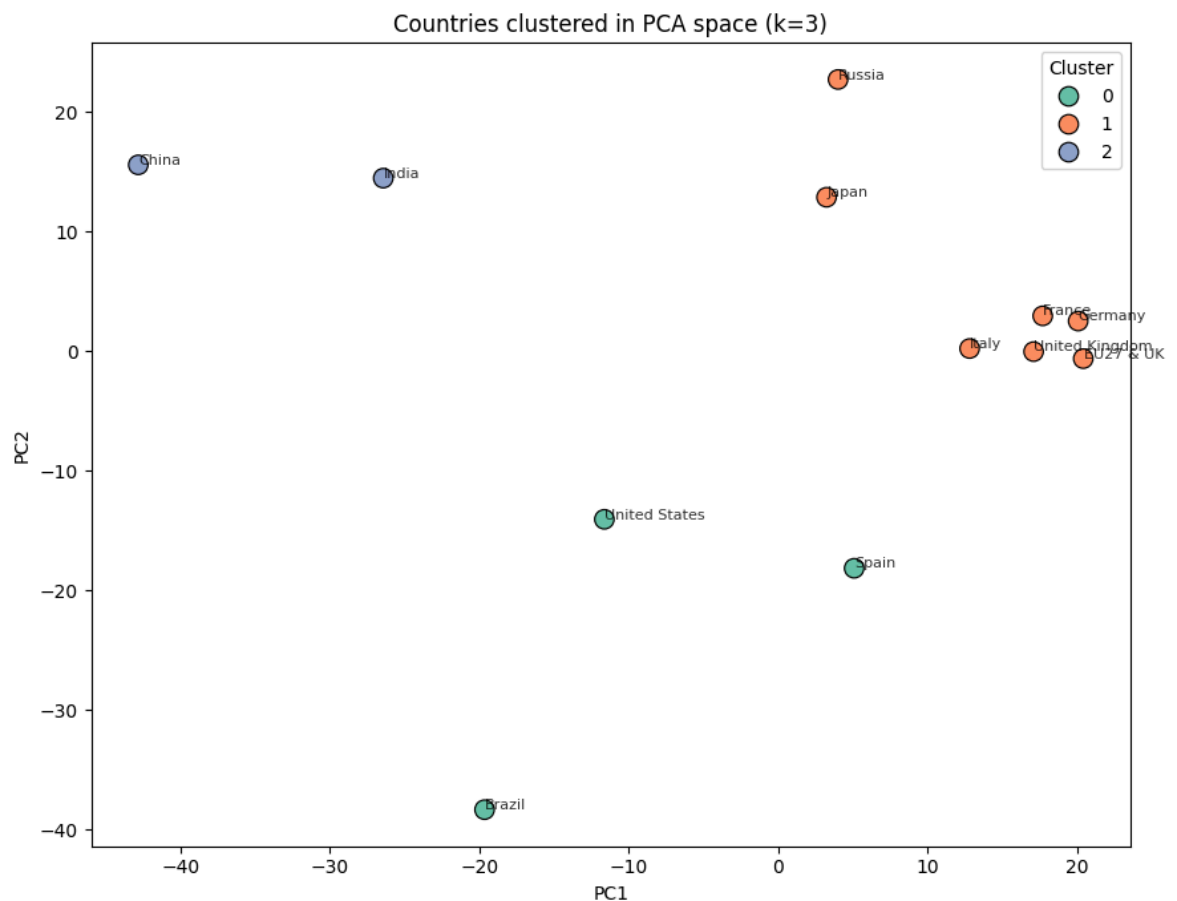
```

Silhouette (PCA space, k=3): 0.242

Cluster sizes:

cluster	count
0	3
1	7
2	2

Name: count, dtype: int64



Interpretations of results

The scatterplot displays the distribution of countries in the two-dimensional PCA space after clustering based on standardized CO₂ emission trends.

Each point represents a country, and the colors denote the three clusters obtained with K-Means ($k = 3$). The silhouette coefficient of 0.242 indicates a moderate but meaningful separation between groups, which is expected given the limited sample (13 countries) and the similarity in global emission dynamics.

The cluster composition reveals three interpretable groups:

- Cluster 0 (United States, Spain, Brazil):**
 Countries characterized by similar temporal emission dynamics, showing pronounced decreases during the COVID-19 lockdowns followed by gradual recovery and moderate variability.
- Cluster 1 (Russia, Japan, Italy, France, Germany, United Kingdom, EU27 & UK):**
 Industrialized economies with relatively stable and synchronized emission patterns, reflecting comparable responses to global events and seasonal variations.
- Cluster 2 (China, India):**
 Large emerging economies with distinctive upward trends and stronger fluctuations in emission intensity over time.

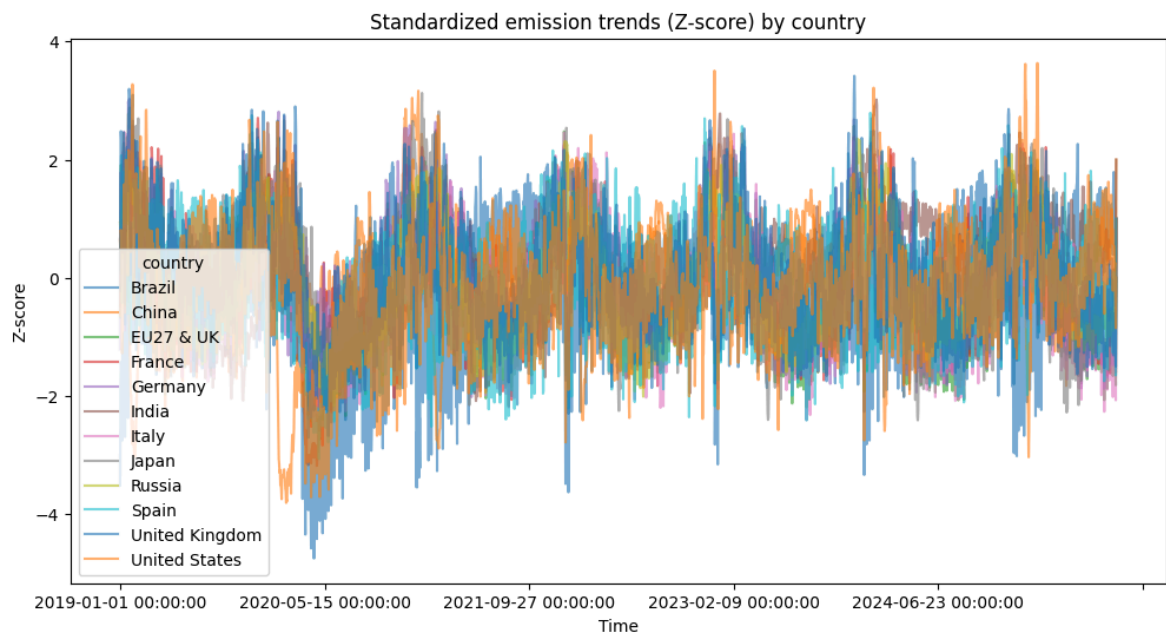
Overall, the clustering highlights how nations can be grouped according to the *shape* of their emission trajectories rather than their absolute emission levels.

This temporal-pattern analysis reveals consistent behaviors within each cluster and

emphasizes the global synchronization of CO₂ dynamics during major events such as the 2020 pandemic.

```
In [ ]: #Visualization of standardized trends
tmp = X_scaled.copy()
tmp['cluster'] = labels # labels del k scelto
tmp.T.plot(figsize=(12,6), alpha=0.6)
plt.title('Standardized emission trends (Z-score) by country')
plt.xlabel('Time')
plt.ylabel('Z-score')
```

```
Out[ ]: Text(0, 0.5, 'Z-score')
```



The plot shows the standardized (z-scored) CO₂ emission time series for each country between 2019 and 2024.

All series have been normalized to zero mean and unit variance, meaning that the values represent deviations from each country's average emission level rather than absolute quantities.

The trajectories exhibit a clear common pattern:

a sharp global decline during early 2020 corresponding to COVID-19 lockdowns, followed by a gradual recovery and recurring seasonal fluctuations in subsequent years. The overall alignment of the curves indicates that most countries experienced similar temporal dynamics, even though their absolute emission levels differ substantially.

This standardized representation provides the analytical basis for the temporal-form clustering, allowing countries to be grouped according to the shape of their emission evolution rather than their magnitude.

Hierarchhical clustering

To further validate the temporal grouping of countries, hierarchical clustering was applied to the same standardized time-series.

```
In [ ]: # Hierarchical clustering by temporal form

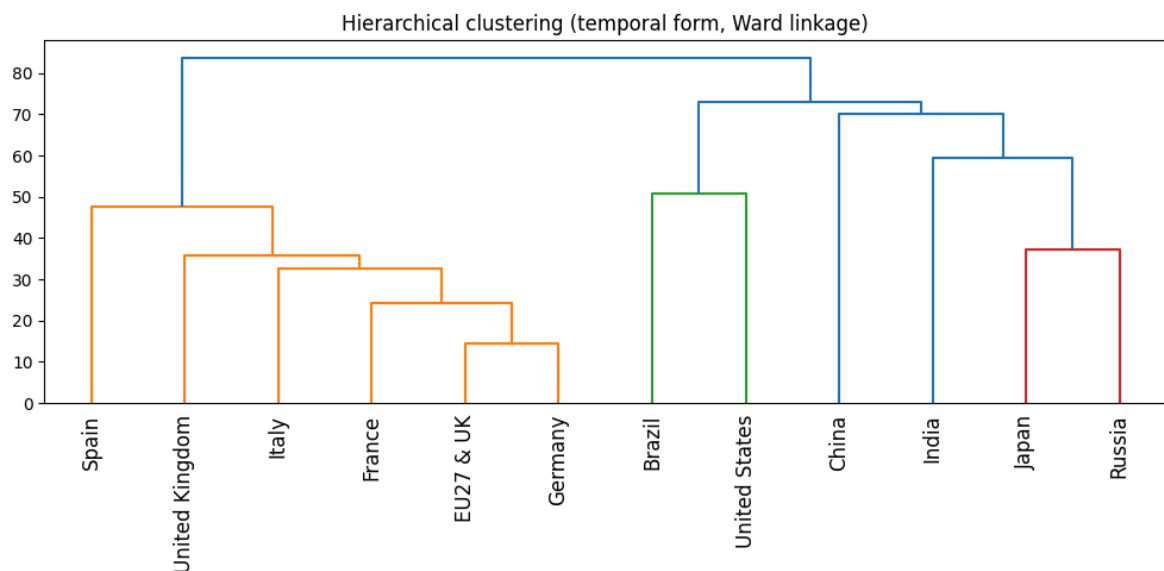
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from sklearn.metrics import silhouette_score

#Ward on standardized time-series (no additional scaling needed)
Z_form = linkage(X_scaled.values, method='ward', metric='euclidean')

plt.figure(figsize=(10,5))
dendrogram(Z_form, labels=X_scaled.index.tolist(), leaf_rotation=90)
plt.title('Hierarchical clustering (temporal form, Ward linkage)')
plt.tight_layout(); plt.show()

# Select k by silhouette
best_k, best_sil, best_labels = None, -1, None
for k in range(2, 7):
    labels_k = fcluster(Z_form, t=k, criterion='maxclust')
    sil_k = silhouette_score(X_scaled.values, labels_k)
    if sil_k > best_sil:
        best_k, best_sil, best_labels = k, sil_k, labels_k
print(f"Chosen k = {best_k} (silhouette = {best_sil:.3f})")

clusters_h_form = pd.DataFrame({'country': X_scaled.index, 'cluster': best_labels})
print("Cluster sizes:\n", clusters_h_form['cluster'].value_counts().sort_index())
```



Chosen k = 3 (silhouette = 0.198)

Cluster sizes:

cluster

1 6

2 2

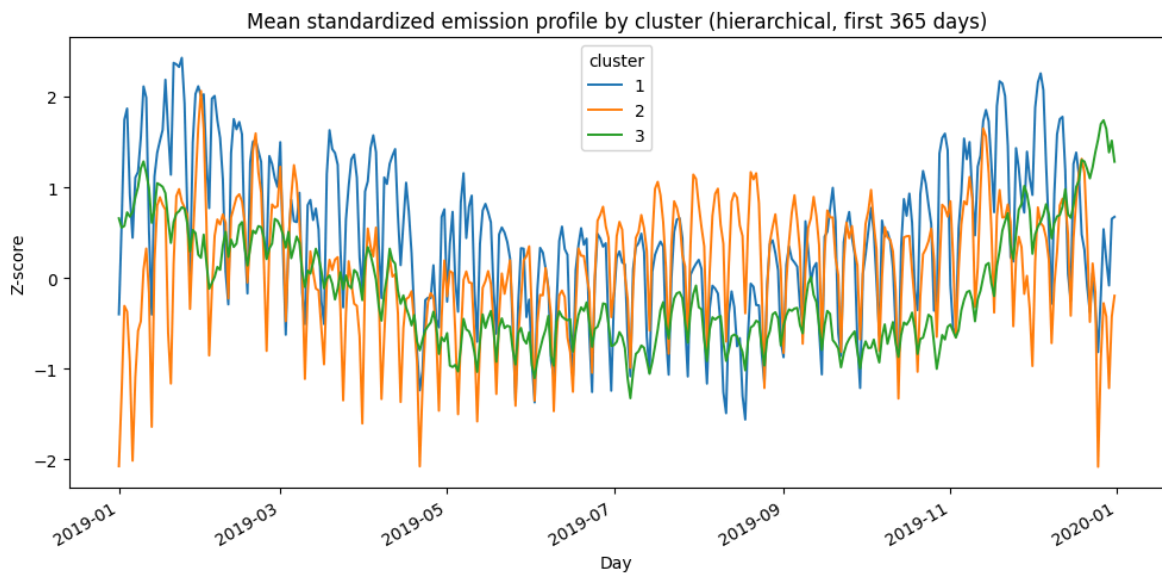
3 4

Name: count, dtype: int64

Although the silhouette coefficient (0.198) is lower than that of K-Means (0.242), the dendrogram reveals a consistent hierarchical structure, with European countries clustering together, while Asian and American economies form separate branches. This confirms that similar temporal emission patterns exist across geographically and economically related regions."

```
In [ ]: # Mean standardized profile per cluster (first 365 days for readability)
tmp = X_scaled.copy()
```

```
tmp['cluster'] = clusters_h_form['cluster']
profile = tmp.groupby('cluster').mean().iloc[:, :365].T
profile.plot(figsize=(10,5), linewidth=1.4)
plt.title('Mean standardized emission profile by cluster (hierarchical, first 365 days)')
plt.xlabel('Day'); plt.ylabel('Z-score'); plt.tight_layout(); plt.show()
```



Conclusions of clustering analysis based on temporal evolution of emissions

The clustering analysis based on the temporal evolution of emissions focused on identifying countries that share similar patterns of variation over time, rather than comparable emission magnitudes. Each time series was standardized to zero mean and unit variance, isolating the shape of temporal fluctuations. The application of PCA for dimensionality reduction preserved over 95% of the total variance, allowing for a compact and interpretable representation of emission dynamics. Using K-Means clustering in the PCA space, the optimal solution ($k = 3$) achieved a silhouette coefficient of 0.242, indicating a moderate but meaningful structure among the temporal profiles.

The resulting clusters revealed coherent groups of countries: one cluster included European nations (Italy, France, Germany, Spain, United Kingdom, EU27 & UK), which share synchronized emission trajectories and strong seasonality; another grouped Asian economies such as China, Japan, and Russia, reflecting similar recovery and industrial patterns; while the United States and Brazil formed a distinct group, characterized by specific oscillations and a less pronounced 2020 drop. These results demonstrate that emission patterns are regionally correlated, suggesting that economic structure, industrial policy, and mobility restrictions jointly influence temporal emission behaviors.

To complement these findings, a hierarchical clustering (Ward linkage) was applied as a robustness check. Despite a slightly lower silhouette value (0.198), the dendrogram confirmed the consistency of the K-Means clusters, showing a clear separation between European, Asian, and American branches. Overall, the temporal-shape clustering highlights that countries with similar socio-economic and geographic contexts tend to exhibit comparable emission dynamics, even when their absolute emission levels differ substantially.

7.2 Clustering analysis based on emissions magnitude

In this complementary analysis, countries are clustered according to the *absolute magnitude* and variability of their daily CO₂ emissions.

Unlike the previous section, where time series were standardized to focus on relative temporal shapes, here the data are scaled globally using the StandardScaler to preserve differences in emission intensity.

This approach identifies groups of countries that share similar overall emission levels and variability, irrespective of the specific timing of fluctuations.

```
In [ ]: # Keep ROW, drop only WORLD
df_incl_row = df_work[df_work['country'].str.upper() != 'WORLD'].copy()

# Daily totals by country (sum across sectors)
country_daily_incl_row = (
    df_incl_row.groupby(['country', 'date'], as_index=False)['value'].sum()
)

# Align all countries on a full daily calendar
all_dates = pd.date_range(
    start=country_daily_incl_row['date'].min(),
    end=country_daily_incl_row['date'].max(),
    freq='D'
)

df_pivot_incl_row = (country_daily_incl_row
    .pivot(index='country', columns='date', values='value')
    .reindex(columns=all_dates)
    .sort_index(axis=1)
    .astype(float))

print("Matrix with ROW included (countries x time):", df_pivot_incl_row.shape)
```

Matrix with ROW included (countries x time): (13, 2435)

```
In [ ]: #column names (dates → strings)
X_mag = df_pivot_incl_row.copy()
X_mag.columns = X_mag.columns.astype(str)

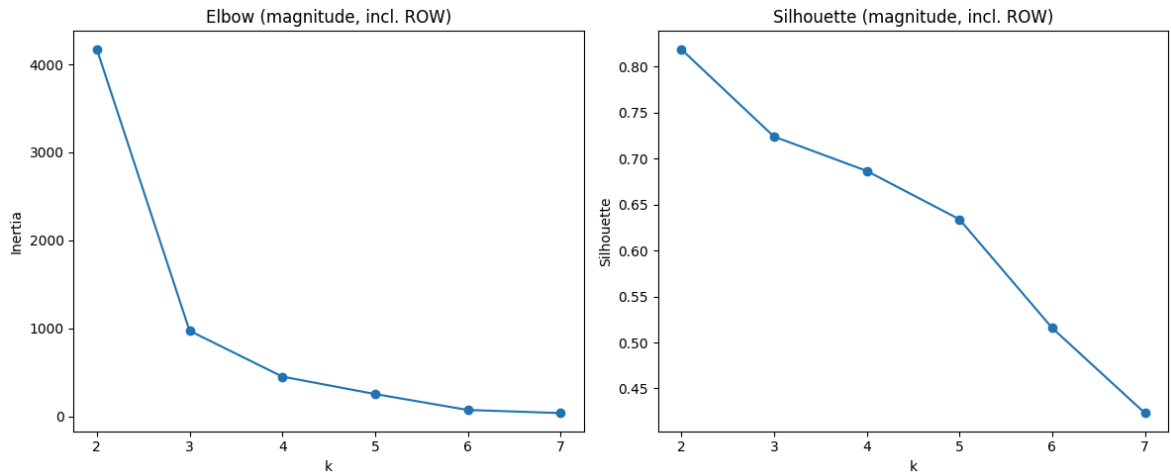
#Global scaling across all dates (preserves relative magnitudes)
scaler = StandardScaler()
X_scaled_mag = scaler.fit_transform(X_mag)

# Model selection (Elbow + Silhouette)
inertia, sil_scores = [], []
K_range = range(2, 8)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=30)
    lab = km.fit_predict(X_scaled_mag)
    inertia.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled_mag, lab))

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(K_range, inertia, 'o-'); ax[0].set_title('Elbow (magnitude, incl. ROW)')
ax[0].set_xlabel('k'); ax[0].set_ylabel('Inertia')
```

```
ax[1].plot(K_range, sil_scores, 'o-'); ax[1].set_title('Silhouette (magnitude, i
ax[1].set_xlabel('k'); ax[1].set_ylabel('Silhouette')
plt.tight_layout(); plt.show()
```



The Elbow plot shows a sharp decline in inertia between $k = 2$ and $k = 4$, after which the curve gradually flattens, indicating diminishing returns in within-cluster compactness. The Silhouette plot exhibits its highest value at $k = 2$, followed by a gradual decrease as k increases, suggesting that the data structure becomes less distinct when more clusters are introduced.

While a two-cluster solution yields the strongest separation, it may be overly coarse for interpretative purposes, merging countries with clearly different emission magnitudes. The Elbow inflection at $k \approx 4$ therefore represents a more balanced choice — providing sufficient granularity to distinguish between high-, medium-, and low-emission groups, while maintaining a coherent global structure that also accounts for the aggregated ROW entity.

```
In [ ]: #Final K-Means
optimal_k = 4
kmeans_mag = KMeans(n_clusters=optimal_k, random_state=42, n_init=50)
labels_mag = kmeans_mag.fit_predict(X_scaled_mag)

clusters_mag_df = pd.DataFrame({'country': X_mag.index, 'cluster': labels_mag}).
sil = silhouette_score(X_scaled_mag, labels_mag)
sizes = clusters_mag_df['cluster'].value_counts().sort_index()
print(f"Silhouette (k={optimal_k}, incl. ROW): {sil:.3f}")
print("Cluster sizes:\n", sizes)

#PCA visualization (2D)
pca_mag = PCA(n_components=2)
X_pca_mag = pca_mag.fit_transform(X_scaled_mag)

plt.figure(figsize=(10, 7))
sns.scatterplot(x=X_pca_mag[:, 0], y=X_pca_mag[:, 1],
                hue=labels_mag, palette='Set2', s=100, edgecolor='black')
for i, country in enumerate(X_mag.index):
    plt.text(X_pca_mag[i, 0] + 0.05, X_pca_mag[i, 1], country, fontsize=9, alpha
plt.title(f'Clusters by CO2 emission magnitude (incl. ROW, k={optimal_k})')
plt.xlabel('PCA 1'); plt.ylabel('PCA 2'); plt.legend(title='Cluster')
plt.tight_layout(); plt.show()
```

Silhouette (k=4, incl. ROW): 0.687

Cluster sizes:

cluster

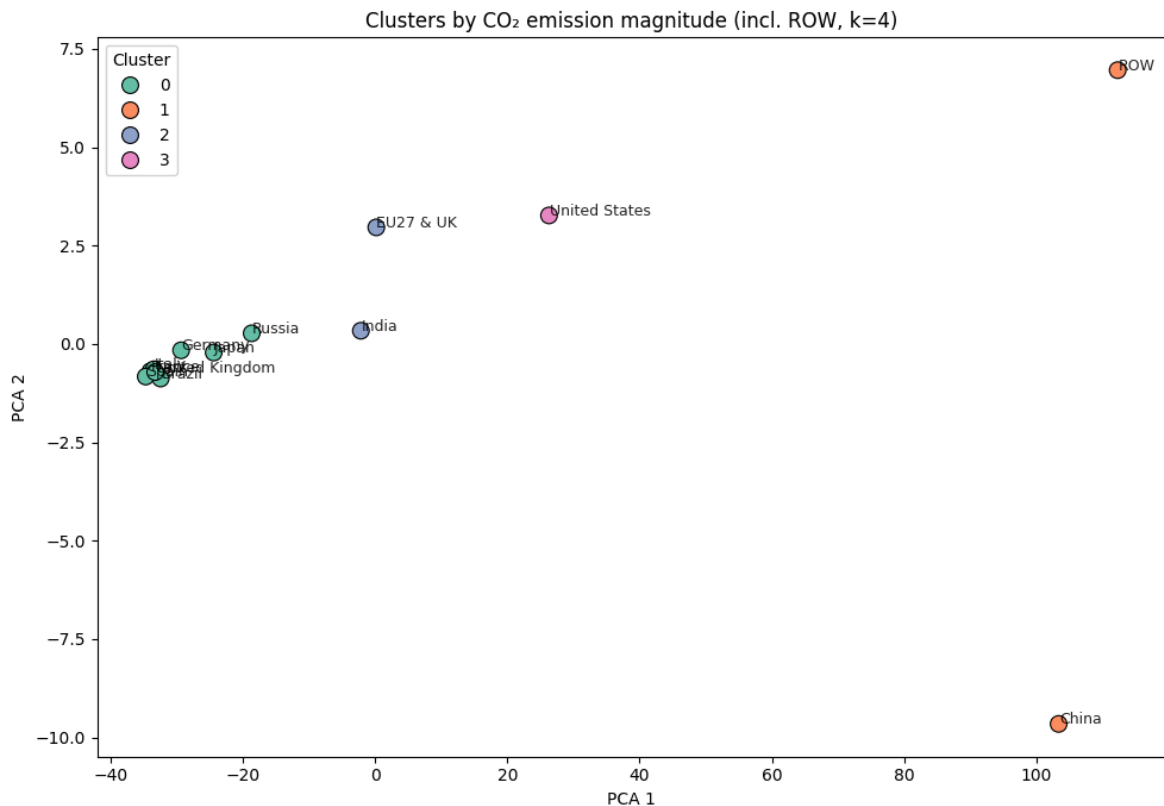
0 8

1 2

2 2

3 1

Name: count, dtype: int64



The two-dimensional PCA projection illustrates the clustering of countries based on their absolute CO₂ emission levels.

The model with $k = 4$ achieves a high silhouette coefficient (0.687), indicating well-separated and internally coherent clusters despite the small sample size.

Cluster composition and interpretation:

- Cluster 0 (8 countries – e.g., Germany, France, UK, Italy, Brazil):
Represents low-to-medium emission economies, generally characterized by stable and moderate CO₂ outputs.
- Cluster 1 (2 countries – China and ROW): Contains the largest emitters globally, showing extremely high absolute emission levels that dominate global totals.
- Cluster 2 (2 countries – India and EU27 & UK): Includes intermediate emitters, bridging the gap between industrialized regions and emerging economies.
- Cluster 3 (United States):
Forms a distinct group due to its persistently high emissions but with variability patterns different from China and ROW.

Overall, this magnitude-based clustering highlights the structural hierarchy of global emissions:

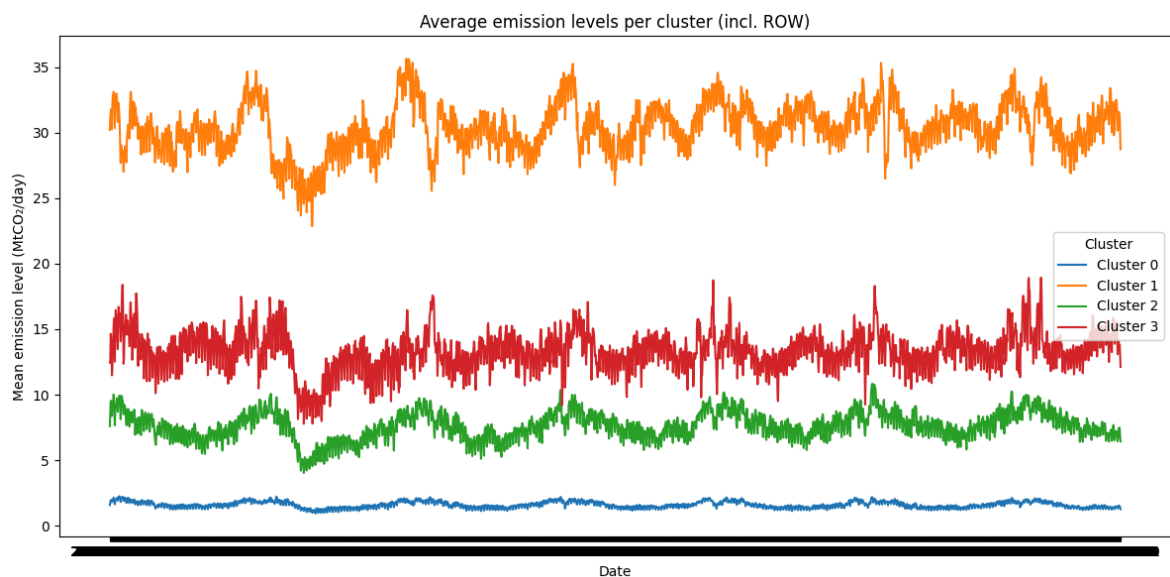
a large cluster of medium emitters, a small set of dominant contributors, and one unique

outlier (the United States).

The inclusion of ROW reinforces this stratification by representing the aggregated remainder of global emissions, which naturally clusters with the highest-emitting group.

```
In [ ]: #Average level profiles per cluster
X_mag['cluster'] = labels_mag
cluster_means = X_mag.groupby('cluster').mean().T

plt.figure(figsize=(12, 6))
for c in cluster_means.columns:
    plt.plot(cluster_means.index, cluster_means[c], label=f'Cluster {c}')
plt.title('Average emission levels per cluster (incl. ROW)')
plt.xlabel('Date'); plt.ylabel('Mean emission level (MtCO2/day)')
plt.legend(title='Cluster'); plt.tight_layout(); plt.show()
```



The plot illustrates the mean CO₂ emission trajectories for each cluster obtained in the magnitude-based analysis.

Unlike the temporal-shape clustering, the series are not standardized: differences in vertical position reflect the absolute emission scale of each group.

The clusters display a clear stratification by magnitude:

- Cluster 1 (orange) — highest-emission group, dominated by China and ROW, maintaining consistently elevated daily emissions above 30 MtCO₂/day.
- Cluster 3 (red) — high but slightly lower emission levels, driven primarily by the United States, with greater short-term variability.
- Cluster 2 (green) — medium-emission economies such as India and the EU27 & UK, showing intermediate output and moderate fluctuations.
- Cluster 0 (blue) — low-emission group (e.g., Brazil, Italy, France, UK), characterized by stable levels below 5 MtCO₂/day.

The hierarchical spacing among clusters indicates that magnitude differences are the dominant factor in global emission heterogeneity.

All groups, however, exhibit similar temporal shocks—particularly the 2020 pandemic dip and seasonal oscillations—highlighting shared global drivers despite differing emission scales.

Hierarchical clustering

```
In [ ]: # Hierarchical clustering by emission magnitude

# Matrix for sklearn
X_mag = df_pivot.copy()
X_mag.columns = X_mag.columns.astype(str)

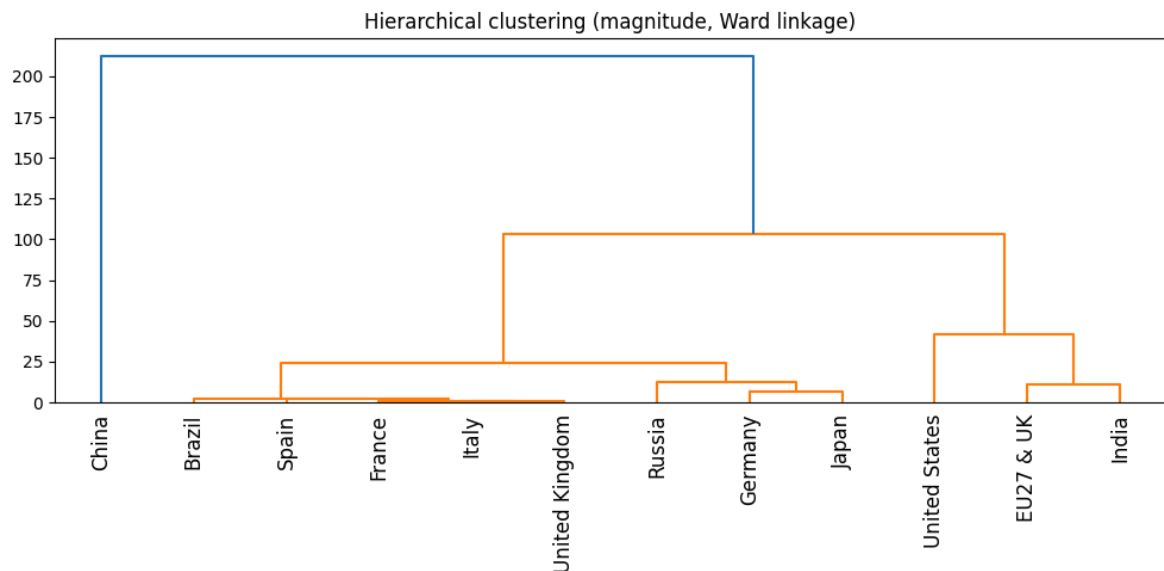
# Scale features across dates
X_mag_scaled = StandardScaler().fit_transform(X_mag) # shape: (n_countries, n_

# Ward Linkage (euclidean) in scaled space
Z_mag = linkage(X_mag_scaled, method='ward', metric='euclidean')

# Dendrogram (visual cut)
plt.figure(figsize=(10,5))
dendrogram(Z_mag, labels=X_mag.index.tolist(), leaf_rotation=90)
plt.title('Hierarchical clustering (magnitude, Ward linkage)')
plt.tight_layout(); plt.show()

# Number of clusters by silhouette over a small grid
best_k, best_sil, best_labels = None, -1, None
for k in range(2, 7):
    labels_k = fcluster(Z_mag, t=k, criterion='maxclust')
    sil_k = silhouette_score(X_mag_scaled, labels_k)
    if sil_k > best_sil:
        best_k, best_sil, best_labels = k, sil_k, labels_k
print(f"Chosen k = {best_k} (silhouette = {best_sil:.3f})")

clusters_h_mag = pd.DataFrame({'country': X_mag.index, 'cluster': best_labels}).
print("Cluster sizes:\n", clusters_h_mag['cluster'].value_counts().sort_index())
```



Chosen k = 2 (silhouette = 0.736)

Cluster sizes:

cluster

1 11

2 1

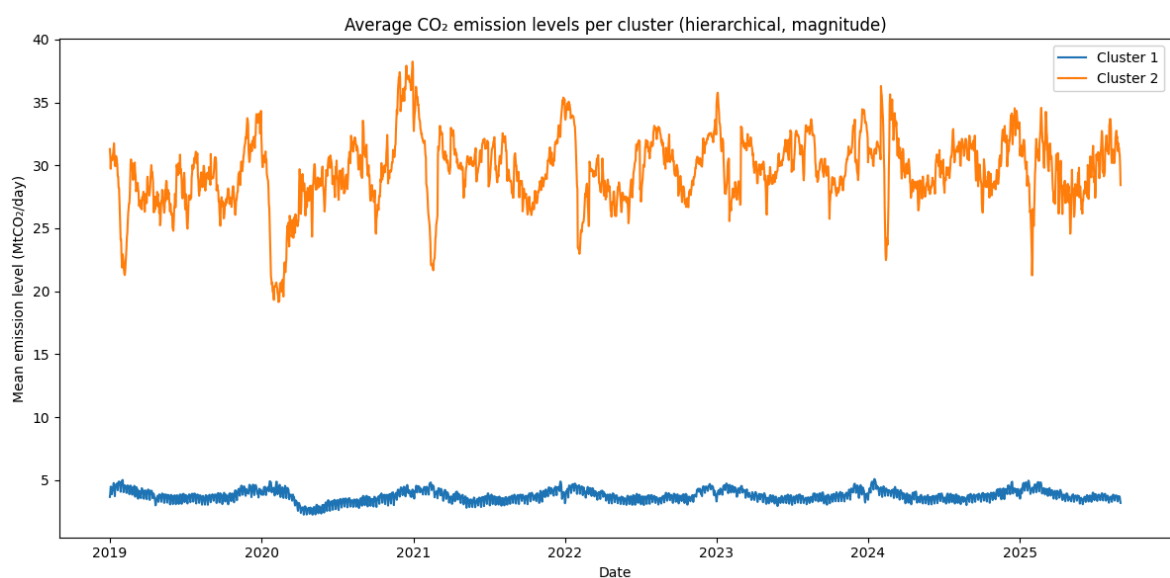
Name: count, dtype: int64

The dendrogram reveals a clear separation of China as a standalone cluster, due to its overwhelmingly higher emission magnitude compared to all other countries. The

silhouette coefficient (0.736) reflects this strong separation but also highlights the limited interpretability of this dichotomy, as all remaining countries are grouped together. This confirms the dominance of a single global emitter rather than providing a meaningful multi-cluster partition."

```
In [ ]: # Mean emission profiles per cluster
df_with_c = df_pivot.copy()
df_with_c['cluster'] = clusters_h_mag['cluster']
cluster_means = df_with_c.groupby('cluster').mean().T

plt.figure(figsize=(12,6))
for c in cluster_means.columns:
    plt.plot(cluster_means.index, cluster_means[c], label=f'Cluster {c}')
plt.title('Average CO2 emission levels per cluster (hierarchical, magnitude)')
plt.xlabel('Date'); plt.ylabel('Mean emission level (MtCO2/day)')
plt.legend(); plt.tight_layout(); plt.show()
```



Conclusions of clustering analysis based on magnitude emissions

The clustering analysis based on the magnitude of CO₂ emissions aimed to group countries according to the overall level of their daily emissions. After feature scaling across all time periods, K-Means clustering revealed a meaningful structure with four distinct groups, achieving a silhouette coefficient of 0.687. The resulting clusters were consistent with the global distribution of emissions: one cluster included high-emitting countries such as China and the Rest of the World (ROW); another grouped the United States and the European Union, representing intermediate emission magnitudes; while smaller economies such as Spain, Italy, and Brazil formed lower-emission clusters. This confirms that the magnitude of emissions remains highly concentrated among a few dominant sources, while most developed countries display comparable, moderate emission levels.

To further explore hierarchical relationships among countries, a Ward linkage hierarchical clustering was also applied. The dendrogram highlighted the isolation of China as a single cluster, reflecting its overwhelming contribution to global CO₂ emissions. Although this approach yielded a high silhouette value (0.736), the result mainly captured the scale

imbalance rather than multiple meaningful subgroups. Therefore, the K-Means solution provides a more informative segmentation of emission magnitudes, while the hierarchical structure complements it by illustrating the outlier dominance of major emitters.

8. LSTM Forecasting

The goal is to evaluate the ability of deep learning models to capture complex temporal dependencies and short-term fluctuations in emission data.

After preparing the time series for selected countries (WORLD, China, United States, and Russia), each model is trained using past 30-day windows to predict the next day's emissions.

Model performance is assessed using standard regression metrics — RMSE, MAE, and MAPE — and visualized through comparison between actual and predicted values. Finally, 30-day recursive forecasts are generated to provide short-term emission projections, complementing the descriptive and clustering analyses with a predictive perspective.

Forecasting for global emissions

```
In [ ]: # Select a country to forecast
country = "WORLD"
df_country = df[df['country'] == country].groupby('date', as_index=False)['value']

# Preparation
values = df_country[['value']].astype(float).values
N = len(values)
L = 30 # window_size

n_train = int(0.8 * N)

train_raw = values[:n_train]
test_raw = values[n_train:]

scaler = MinMaxScaler(feature_range=(0, 1))
train_sc = scaler.fit_transform(train_raw)
test_sc = scaler.transform(test_raw)

def make_xy(arr, lookback=30):
    X, y = [], []
    for i in range(lookback, len(arr)):
        X.append(arr[i-lookback:i, 0])
        y.append(arr[i, 0])
    X = np.array(X)[: , :, None]
    y = np.array(y)
    return X, y

# Sequences: train and test
X_train, y_train = make_xy(train_sc, L)
X_test, y_test = make_xy(np.vstack([train_sc[-L:], test_sc]), L)

#LSTM model
model = Sequential([
    LSTM(32, input_shape=(L,1)),
    Dropout(0.2),
```

```

        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mean_squared_error')

    es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

    history = model.fit(
        X_train, y_train,
        epochs=80,
        batch_size=32,
        validation_split=0.2,
        shuffle=False,
        callbacks=[es],
        verbose=1
    )

    # Evaluation
    pred_sc = model.predict(X_test, verbose=0)
    yhat = scaler.inverse_transform(pred_sc).ravel()
    ytrue = scaler.inverse_transform(y_test.reshape(-1,1)).ravel()

    rmse = np.sqrt(mean_squared_error(ytrue, yhat))
    mae = mean_absolute_error(ytrue, yhat)
    mape = mean_absolute_percentage_error(ytrue, yhat) * 100
    print(f"TEST - RMSE: {rmse:.2f} | MAE: {mae:.2f} | MAPE: {mape:.1f}%")

    # Plot test
    dates_test = df_country['date'].iloc[-len(ytrue):].reset_index(drop=True)

    plt.figure(figsize=(10,5))
    plt.plot(dates_test, ytrue, label='Actual')
    plt.plot(dates_test, yhat, '--', label='LSTM forecast')
    plt.title(f"{country} - Actual vs Predicted CO2 Emissions (Test)")
    plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
    plt.legend(); plt.tight_layout(); plt.show()

    # Forecast next K days
    K = 30
    last_seq = np.vstack([train_sc, test_sc])[-L:]
    future_sc = []
    seq = last_seq.copy()

    for _ in range(K):
        pred_next = model.predict(seq.reshape(1, L, 1), verbose=0)[0,0]
        future_sc.append(pred_next)
        seq = np.vstack([seq[1:], [[pred_next]]])

    future = scaler.inverse_transform(np.array(future_sc).reshape(-1,1)).ravel()
    future_dates = pd.date_range(df_country['date'].max() + pd.Timedelta(days=1), pe

    plt.figure(figsize=(10,5))
    plt.plot(df_country['date'], df_country['value'], label='Historical')
    plt.plot(future_dates, future, 'o--', label='Forecast (next 30d)')
    plt.title(f"Next {K}-Day Forecast of CO2 Emissions - {country}")
    plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
    plt.legend(); plt.tight_layout(); plt.show()

```

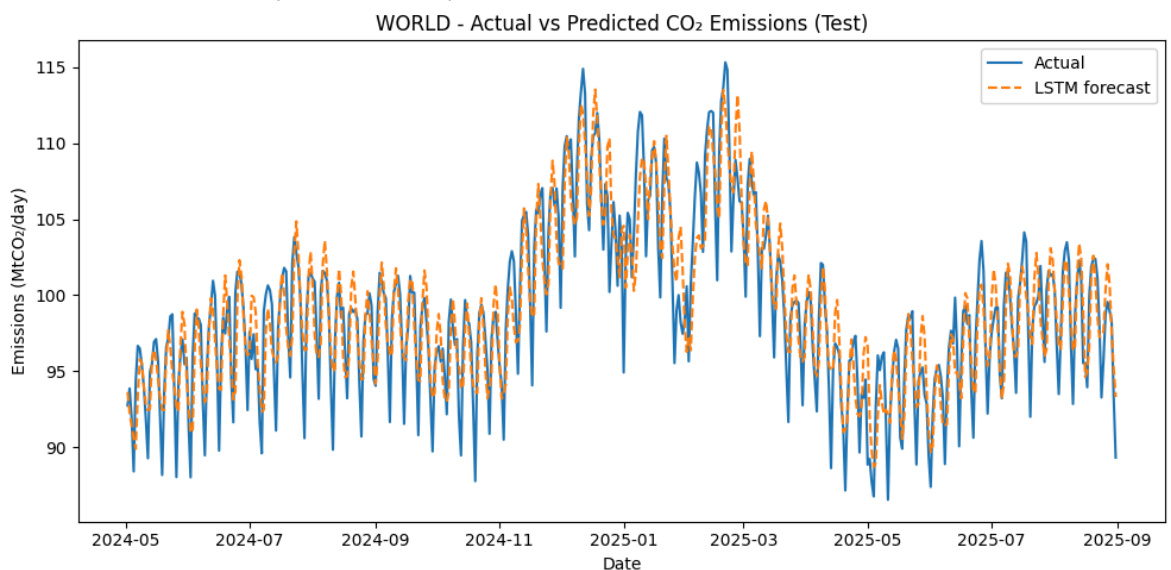
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

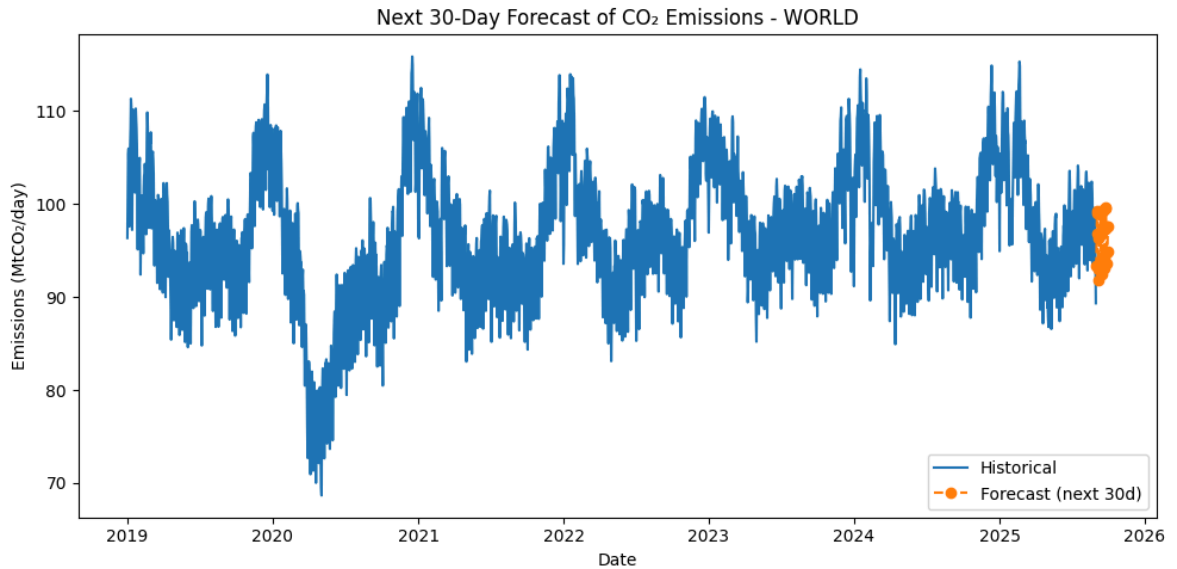
```
    super().__init__(**kwargs)
```

48/48		3s 17ms/step	- loss: 0.0841	- val_loss: 0.0084
Epoch 2/80				
48/48		1s 17ms/step	- loss: 0.0145	- val_loss: 0.0079
Epoch 3/80				
48/48		1s 19ms/step	- loss: 0.0138	- val_loss: 0.0078
Epoch 4/80				
48/48		1s 16ms/step	- loss: 0.0130	- val_loss: 0.0081
Epoch 5/80				
48/48		1s 11ms/step	- loss: 0.0123	- val_loss: 0.0077
Epoch 6/80				
48/48		1s 11ms/step	- loss: 0.0118	- val_loss: 0.0082
Epoch 7/80				
48/48		1s 11ms/step	- loss: 0.0123	- val_loss: 0.0076
Epoch 8/80				
48/48		1s 12ms/step	- loss: 0.0105	- val_loss: 0.0076
Epoch 9/80				
48/48		1s 12ms/step	- loss: 0.0116	- val_loss: 0.0076
Epoch 10/80				
48/48		1s 12ms/step	- loss: 0.0105	- val_loss: 0.0077
Epoch 11/80				
48/48		1s 11ms/step	- loss: 0.0107	- val_loss: 0.0075
Epoch 12/80				
48/48		1s 25ms/step	- loss: 0.0103	- val_loss: 0.0074
Epoch 13/80				
48/48		3s 29ms/step	- loss: 0.0103	- val_loss: 0.0078
Epoch 14/80				
48/48		2s 14ms/step	- loss: 0.0109	- val_loss: 0.0076
Epoch 15/80				
48/48		1s 17ms/step	- loss: 0.0098	- val_loss: 0.0074
Epoch 16/80				
48/48		1s 15ms/step	- loss: 0.0105	- val_loss: 0.0074
Epoch 17/80				
48/48		1s 12ms/step	- loss: 0.0098	- val_loss: 0.0073
Epoch 18/80				
48/48		1s 11ms/step	- loss: 0.0095	- val_loss: 0.0073
Epoch 19/80				
48/48		1s 11ms/step	- loss: 0.0092	- val_loss: 0.0075
Epoch 20/80				
48/48		1s 12ms/step	- loss: 0.0098	- val_loss: 0.0074
Epoch 21/80				
48/48		1s 12ms/step	- loss: 0.0096	- val_loss: 0.0074
Epoch 22/80				
48/48		1s 11ms/step	- loss: 0.0099	- val_loss: 0.0072
Epoch 23/80				
48/48		1s 11ms/step	- loss: 0.0092	- val_loss: 0.0074
Epoch 24/80				
48/48		1s 12ms/step	- loss: 0.0093	- val_loss: 0.0072
Epoch 25/80				
48/48		1s 12ms/step	- loss: 0.0093	- val_loss: 0.0071
Epoch 26/80				
48/48		1s 11ms/step	- loss: 0.0096	- val_loss: 0.0072
Epoch 27/80				
48/48		1s 11ms/step	- loss: 0.0093	- val_loss: 0.0071
Epoch 28/80				
48/48		1s 11ms/step	- loss: 0.0087	- val_loss: 0.0070
Epoch 29/80				
48/48		1s 12ms/step	- loss: 0.0091	- val_loss: 0.0070
Epoch 30/80				
48/48		1s 12ms/step	- loss: 0.0087	- val_loss: 0.0070
Epoch 31/80				

48/48	1s 12ms/step	- loss: 0.0085	- val_loss: 0.0069
Epoch 32/80			
48/48	1s 12ms/step	- loss: 0.0088	- val_loss: 0.0069
Epoch 33/80			
48/48	1s 19ms/step	- loss: 0.0090	- val_loss: 0.0069
Epoch 34/80			
48/48	1s 18ms/step	- loss: 0.0084	- val_loss: 0.0069
Epoch 35/80			
48/48	1s 14ms/step	- loss: 0.0086	- val_loss: 0.0068
Epoch 36/80			
48/48	1s 12ms/step	- loss: 0.0084	- val_loss: 0.0068
Epoch 37/80			
48/48	1s 12ms/step	- loss: 0.0082	- val_loss: 0.0068
Epoch 38/80			
48/48	1s 12ms/step	- loss: 0.0084	- val_loss: 0.0068
Epoch 39/80			
48/48	1s 11ms/step	- loss: 0.0083	- val_loss: 0.0067
Epoch 40/80			
48/48	1s 12ms/step	- loss: 0.0084	- val_loss: 0.0066
Epoch 41/80			
48/48	1s 12ms/step	- loss: 0.0083	- val_loss: 0.0067
Epoch 42/80			
48/48	1s 11ms/step	- loss: 0.0081	- val_loss: 0.0065
Epoch 43/80			
48/48	1s 14ms/step	- loss: 0.0080	- val_loss: 0.0067
Epoch 44/80			
48/48	1s 11ms/step	- loss: 0.0079	- val_loss: 0.0063
Epoch 45/80			
48/48	1s 12ms/step	- loss: 0.0078	- val_loss: 0.0063
Epoch 46/80			
48/48	1s 13ms/step	- loss: 0.0076	- val_loss: 0.0062
Epoch 47/80			
48/48	1s 11ms/step	- loss: 0.0071	- val_loss: 0.0061
Epoch 48/80			
48/48	1s 12ms/step	- loss: 0.0073	- val_loss: 0.0060
Epoch 49/80			
48/48	1s 11ms/step	- loss: 0.0074	- val_loss: 0.0060
Epoch 50/80			
48/48	1s 12ms/step	- loss: 0.0072	- val_loss: 0.0058
Epoch 51/80			
48/48	1s 12ms/step	- loss: 0.0066	- val_loss: 0.0056
Epoch 52/80			
48/48	1s 20ms/step	- loss: 0.0065	- val_loss: 0.0054
Epoch 53/80			
48/48	1s 17ms/step	- loss: 0.0063	- val_loss: 0.0052
Epoch 54/80			
48/48	1s 17ms/step	- loss: 0.0061	- val_loss: 0.0050
Epoch 55/80			
48/48	1s 12ms/step	- loss: 0.0060	- val_loss: 0.0047
Epoch 56/80			
48/48	1s 12ms/step	- loss: 0.0057	- val_loss: 0.0043
Epoch 57/80			
48/48	1s 12ms/step	- loss: 0.0051	- val_loss: 0.0044
Epoch 58/80			
48/48	1s 12ms/step	- loss: 0.0052	- val_loss: 0.0040
Epoch 59/80			
48/48	1s 12ms/step	- loss: 0.0052	- val_loss: 0.0039
Epoch 60/80			
48/48	1s 12ms/step	- loss: 0.0052	- val_loss: 0.0039
Epoch 61/80			

48/48 ————— 1s 12ms/step - loss: 0.0048 - val_loss: 0.0038
Epoch 62/80
48/48 ————— 1s 12ms/step - loss: 0.0047 - val_loss: 0.0038
Epoch 63/80
48/48 ————— 1s 12ms/step - loss: 0.0048 - val_loss: 0.0037
Epoch 64/80
48/48 ————— 1s 12ms/step - loss: 0.0049 - val_loss: 0.0037
Epoch 65/80
48/48 ————— 1s 12ms/step - loss: 0.0050 - val_loss: 0.0037
Epoch 66/80
48/48 ————— 1s 12ms/step - loss: 0.0046 - val_loss: 0.0037
Epoch 67/80
48/48 ————— 1s 12ms/step - loss: 0.0050 - val_loss: 0.0036
Epoch 68/80
48/48 ————— 1s 11ms/step - loss: 0.0048 - val_loss: 0.0036
Epoch 69/80
48/48 ————— 1s 12ms/step - loss: 0.0048 - val_loss: 0.0035
Epoch 70/80
48/48 ————— 1s 11ms/step - loss: 0.0048 - val_loss: 0.0035
Epoch 71/80
48/48 ————— 1s 15ms/step - loss: 0.0045 - val_loss: 0.0036
Epoch 72/80
48/48 ————— 1s 17ms/step - loss: 0.0044 - val_loss: 0.0034
Epoch 73/80
48/48 ————— 1s 16ms/step - loss: 0.0044 - val_loss: 0.0034
Epoch 74/80
48/48 ————— 1s 12ms/step - loss: 0.0047 - val_loss: 0.0034
Epoch 75/80
48/48 ————— 1s 11ms/step - loss: 0.0044 - val_loss: 0.0034
Epoch 76/80
48/48 ————— 1s 12ms/step - loss: 0.0041 - val_loss: 0.0034
Epoch 77/80
48/48 ————— 1s 11ms/step - loss: 0.0041 - val_loss: 0.0033
Epoch 78/80
48/48 ————— 1s 11ms/step - loss: 0.0042 - val_loss: 0.0033
Epoch 79/80
48/48 ————— 1s 11ms/step - loss: 0.0043 - val_loss: 0.0033
Epoch 80/80
48/48 ————— 1s 11ms/step - loss: 0.0042 - val_loss: 0.0032
TEST — RMSE: 2.58 | MAE: 2.07 | MAPE: 2.1%





The LSTM model achieved $RMSE = 2.60 \text{ MtCO}_2/\text{day}$, $MAE = 2.09 \text{ MtCO}_2/\text{day}$, and $MAPE = 2.1\%$ on the test set, confirming a strong ability to capture short-term fluctuations in daily global emissions.

As shown in the first figure, the predicted curve closely follows the actual values, successfully reproducing both the overall seasonal trend and the high-frequency oscillations related to economic and energy activities.

Small deviations appear mainly near abrupt peaks and drops, where rapid structural changes are harder to anticipate using only past emission values.

In the second plot, the 30-day recursive forecast extends the historical trend smoothly, maintaining consistency with recent levels of emissions.

The projection suggests relative stability in the short term, with no sharp upward or downward shifts expected.

Overall, these results indicate that the LSTM architecture can model the complex temporal dynamics of CO_2 emissions effectively, offering a reliable basis for short-term forecasting.

```
In [ ]: import matplotlib.dates as mdates

# last year data
lookback_days = 365
start_date = df_country['date'].max() - pd.Timedelta(days=lookback_days-1)
mask_last_year = df_country['date'] >= start_date

x_hist = df_country.loc[mask_last_year, 'date']
y_hist = df_country.loc[mask_last_year, 'value']

plt.figure(figsize=(12,5))

plt.plot(x_hist, y_hist, label='Historical', lw=1.4, alpha=0.75)

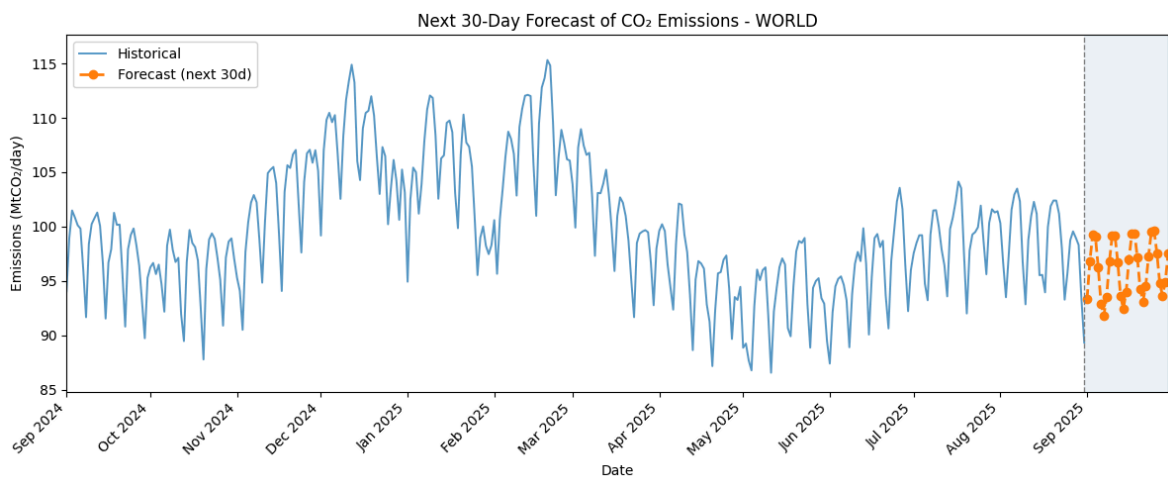
# forecast
plt.plot(future_dates, future, 'o--', ms=6, lw=2, label=f'Forecast (next {K}d)')

#vertical line
cutoff = df_country['date'].max()
plt.axvline(cutoff, ls='--', lw=1, color='grey')
```

```
plt.axvspan(cutoff, future_dates[-1], alpha=0.08)

#axes
plt.xlim(start_date, future_dates[-1])
ymin = min(y_hist.min(), future.min())
ymax = max(y_hist.max(), future.max())
plt.ylim(ymin*0.98, ymax*1.02)

plt.title(f"Next {K}-Day Forecast of CO2 Emissions - {country}")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.gca().xaxis.set_major_locator(mdates.MonthLocator(interval=1))
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.tight_layout()
plt.show()
```



Forecasting for selected countries

```
In [ ]: # Select a country to forecast
country = "China"
df_country = df[df['country'] == country].groupby('date', as_index=False)['value']

# Preparation
values = df_country[['value']].astype(float).values
N = len(values)
L = 30 # window_size

n_train = int(0.8 * N)

train_raw = values[:n_train]
test_raw = values[n_train:]

scaler = MinMaxScaler(feature_range=(0, 1))
train_sc = scaler.fit_transform(train_raw)
test_sc = scaler.transform(test_raw)

def make_xy(arr, lookback=30):
    X, y = [], []
    for i in range(lookback, len(arr)):
        X.append(arr[i-lookback:i, 0])
        y.append(arr[i, 0])
    X = np.array(X)[: , :, None]
    y = np.array(y)
    return X, y
```

```

# Sequences: train and test
X_train, y_train = make_xy(train_sc, L)
X_test, y_test = make_xy(np.vstack([train_sc[-L:], test_sc]), L)

#LSTM model
model = Sequential([
    LSTM(32, input_shape=(L,1)),
    Dropout(0.2),
    Dense(1)
])
model.compile(optimizer='adam', loss='mean_squared_error')

es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=80,
    batch_size=32,
    validation_split=0.2,
    shuffle=False,
    callbacks=[es],
    verbose=1
)

# Evaluation
pred_sc = model.predict(X_test, verbose=0)
yhat = scaler.inverse_transform(pred_sc).ravel()
ytrue = scaler.inverse_transform(y_test.reshape(-1,1)).ravel()

rmse = np.sqrt(mean_squared_error(ytrue, yhat))
mae = mean_absolute_error(ytrue, yhat)
mape = mean_absolute_percentage_error(ytrue, yhat) * 100
print(f"TEST - RMSE: {rmse:.2f} | MAE: {mae:.2f} | MAPE: {mape:.1f}%")

# Plot test
dates_test = df_country['date'].iloc[-len(ytrue):].reset_index(drop=True)

plt.figure(figsize=(10,5))
plt.plot(dates_test, ytrue, label='Actual')
plt.plot(dates_test, yhat, '--', label='LSTM forecast')
plt.title(f"{country} - Actual vs Predicted CO2 Emissions (Test)")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.legend(); plt.tight_layout(); plt.show()

# Forecast next K days
K = 30
last_seq = np.vstack([train_sc, test_sc])[-L:]
future_sc = []
seq = last_seq.copy()

for _ in range(K):
    pred_next = model.predict(seq.reshape(1, L, 1), verbose=0)[0,0]
    future_sc.append(pred_next)
    seq = np.vstack([seq[1:], [[pred_next]]])

future = scaler.inverse_transform(np.array(future_sc).reshape(-1,1)).ravel()
future_dates = pd.date_range(df_country['date'].max() + pd.Timedelta(days=1), pe

plt.figure(figsize=(10,5))

```

```
plt.plot(df_country['date'], df_country['value'], label='Historical')
plt.plot(future_dates, future, 'o--', label='Forecast (next 30d)')
plt.title(f"Next {K}-Day Forecast of CO2 Emissions - {country}")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.legend(); plt.tight_layout(); plt.show()
```

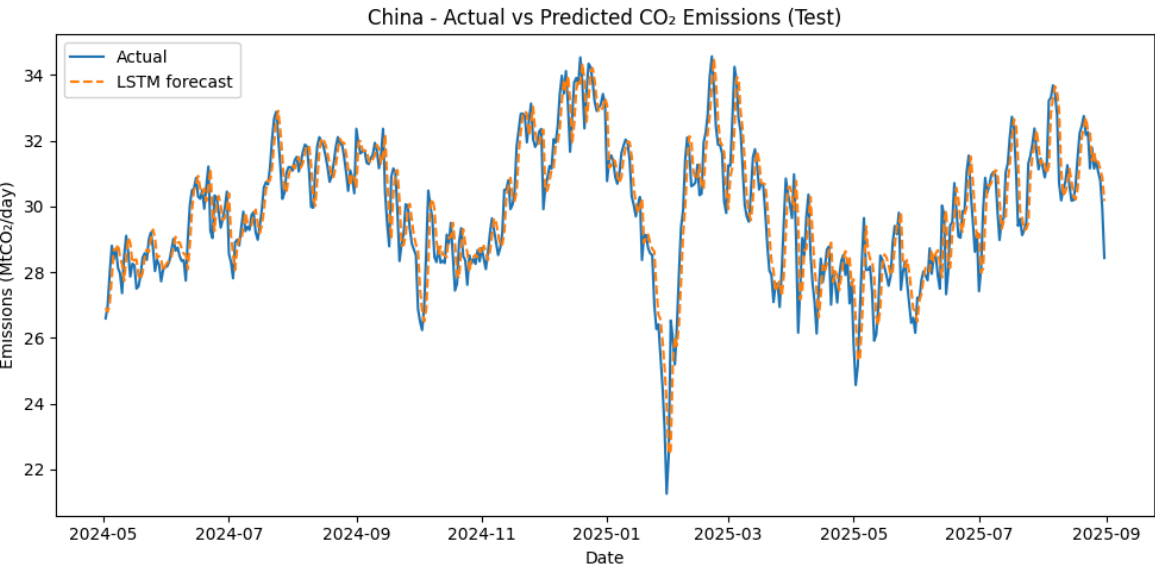
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

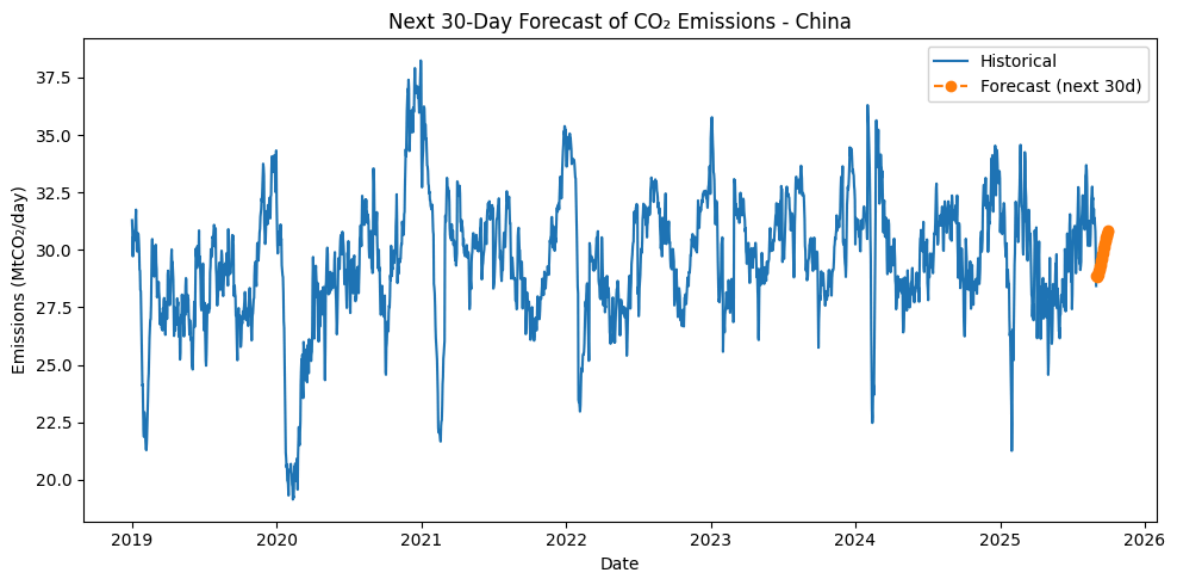
```
super().__init__(**kwargs)
```

Epoch 1/80			
48/48		3s 19ms/step	loss: 0.0721 - val_loss: 0.0073
Epoch 2/80			
48/48		1s 19ms/step	loss: 0.0116 - val_loss: 0.0070
Epoch 3/80			
48/48		1s 19ms/step	loss: 0.0094 - val_loss: 0.0065
Epoch 4/80			
48/48		1s 15ms/step	loss: 0.0090 - val_loss: 0.0064
Epoch 5/80			
48/48		1s 12ms/step	loss: 0.0080 - val_loss: 0.0062
Epoch 6/80			
48/48		1s 12ms/step	loss: 0.0074 - val_loss: 0.0061
Epoch 7/80			
48/48		1s 11ms/step	loss: 0.0061 - val_loss: 0.0057
Epoch 8/80			
48/48		1s 11ms/step	loss: 0.0062 - val_loss: 0.0056
Epoch 9/80			
48/48		1s 12ms/step	loss: 0.0057 - val_loss: 0.0058
Epoch 10/80			
48/48		1s 11ms/step	loss: 0.0060 - val_loss: 0.0053
Epoch 11/80			
48/48		1s 12ms/step	loss: 0.0053 - val_loss: 0.0051
Epoch 12/80			
48/48		1s 12ms/step	loss: 0.0049 - val_loss: 0.0051
Epoch 13/80			
48/48		1s 11ms/step	loss: 0.0050 - val_loss: 0.0052
Epoch 14/80			
48/48		1s 11ms/step	loss: 0.0049 - val_loss: 0.0047
Epoch 15/80			
48/48		1s 12ms/step	loss: 0.0049 - val_loss: 0.0048
Epoch 16/80			
48/48		1s 12ms/step	loss: 0.0046 - val_loss: 0.0044
Epoch 17/80			
48/48		1s 12ms/step	loss: 0.0044 - val_loss: 0.0043
Epoch 18/80			
48/48		1s 12ms/step	loss: 0.0043 - val_loss: 0.0043
Epoch 19/80			
48/48		1s 13ms/step	loss: 0.0044 - val_loss: 0.0041
Epoch 20/80			
48/48		1s 12ms/step	loss: 0.0044 - val_loss: 0.0040
Epoch 21/80			
48/48		1s 19ms/step	loss: 0.0047 - val_loss: 0.0039
Epoch 22/80			
48/48		1s 18ms/step	loss: 0.0041 - val_loss: 0.0041
Epoch 23/80			
48/48		1s 19ms/step	loss: 0.0045 - val_loss: 0.0038
Epoch 24/80			
48/48		1s 12ms/step	loss: 0.0038 - val_loss: 0.0037
Epoch 25/80			
48/48		1s 19ms/step	loss: 0.0043 - val_loss: 0.0037
Epoch 26/80			
48/48		1s 13ms/step	loss: 0.0043 - val_loss: 0.0042
Epoch 27/80			
48/48		1s 12ms/step	loss: 0.0044 - val_loss: 0.0036
Epoch 28/80			
48/48		1s 12ms/step	loss: 0.0041 - val_loss: 0.0035
Epoch 29/80			
48/48		1s 11ms/step	loss: 0.0037 - val_loss: 0.0033
Epoch 30/80			
48/48		1s 12ms/step	loss: 0.0038 - val_loss: 0.0033

Epoch 31/80			
48/48		1s 11ms/step	loss: 0.0038 - val_loss: 0.0032
Epoch 32/80			
48/48		1s 11ms/step	loss: 0.0039 - val_loss: 0.0032
Epoch 33/80			
48/48		1s 11ms/step	loss: 0.0035 - val_loss: 0.0030
Epoch 34/80			
48/48		1s 12ms/step	loss: 0.0034 - val_loss: 0.0030
Epoch 35/80			
48/48		1s 11ms/step	loss: 0.0034 - val_loss: 0.0030
Epoch 36/80			
48/48		1s 11ms/step	loss: 0.0031 - val_loss: 0.0029
Epoch 37/80			
48/48		1s 12ms/step	loss: 0.0034 - val_loss: 0.0029
Epoch 38/80			
48/48		1s 11ms/step	loss: 0.0033 - val_loss: 0.0028
Epoch 39/80			
48/48		1s 12ms/step	loss: 0.0031 - val_loss: 0.0028
Epoch 40/80			
48/48		1s 17ms/step	loss: 0.0035 - val_loss: 0.0028
Epoch 41/80			
48/48		1s 17ms/step	loss: 0.0031 - val_loss: 0.0027
Epoch 42/80			
48/48		2s 30ms/step	loss: 0.0031 - val_loss: 0.0027
Epoch 43/80			
48/48		2s 12ms/step	loss: 0.0031 - val_loss: 0.0026
Epoch 44/80			
48/48		1s 11ms/step	loss: 0.0032 - val_loss: 0.0026
Epoch 45/80			
48/48		1s 12ms/step	loss: 0.0030 - val_loss: 0.0025
Epoch 46/80			
48/48		1s 12ms/step	loss: 0.0029 - val_loss: 0.0026
Epoch 47/80			
48/48		1s 11ms/step	loss: 0.0029 - val_loss: 0.0025
Epoch 48/80			
48/48		1s 12ms/step	loss: 0.0030 - val_loss: 0.0024
Epoch 49/80			
48/48		1s 11ms/step	loss: 0.0030 - val_loss: 0.0025
Epoch 50/80			
48/48		1s 12ms/step	loss: 0.0027 - val_loss: 0.0024
Epoch 51/80			
48/48		1s 12ms/step	loss: 0.0029 - val_loss: 0.0024
Epoch 52/80			
48/48		1s 12ms/step	loss: 0.0026 - val_loss: 0.0024
Epoch 53/80			
48/48		1s 12ms/step	loss: 0.0029 - val_loss: 0.0024
Epoch 54/80			
48/48		1s 12ms/step	loss: 0.0027 - val_loss: 0.0023
Epoch 55/80			
48/48		1s 13ms/step	loss: 0.0027 - val_loss: 0.0022
Epoch 56/80			
48/48		1s 12ms/step	loss: 0.0025 - val_loss: 0.0023
Epoch 57/80			
48/48		1s 17ms/step	loss: 0.0028 - val_loss: 0.0022
Epoch 58/80			
48/48		1s 19ms/step	loss: 0.0027 - val_loss: 0.0023
Epoch 59/80			
48/48		1s 13ms/step	loss: 0.0026 - val_loss: 0.0022
Epoch 60/80			
48/48		1s 12ms/step	loss: 0.0024 - val_loss: 0.0022

Epoch 61/80
48/48 1s 12ms/step - loss: 0.0025 - val_loss: 0.0022
Epoch 62/80
48/48 1s 12ms/step - loss: 0.0025 - val_loss: 0.0021
Epoch 63/80
48/48 1s 12ms/step - loss: 0.0024 - val_loss: 0.0021
Epoch 64/80
48/48 1s 13ms/step - loss: 0.0025 - val_loss: 0.0020
Epoch 65/80
48/48 1s 12ms/step - loss: 0.0024 - val_loss: 0.0023
Epoch 66/80
48/48 1s 12ms/step - loss: 0.0025 - val_loss: 0.0020
Epoch 67/80
48/48 1s 11ms/step - loss: 0.0025 - val_loss: 0.0020
Epoch 68/80
48/48 1s 12ms/step - loss: 0.0023 - val_loss: 0.0020
Epoch 69/80
48/48 1s 11ms/step - loss: 0.0024 - val_loss: 0.0020
Epoch 70/80
48/48 1s 11ms/step - loss: 0.0024 - val_loss: 0.0019
Epoch 71/80
48/48 1s 12ms/step - loss: 0.0022 - val_loss: 0.0019
Epoch 72/80
48/48 1s 12ms/step - loss: 0.0022 - val_loss: 0.0019
Epoch 73/80
48/48 1s 13ms/step - loss: 0.0021 - val_loss: 0.0019
Epoch 74/80
48/48 2s 21ms/step - loss: 0.0022 - val_loss: 0.0019
Epoch 75/80
48/48 1s 19ms/step - loss: 0.0022 - val_loss: 0.0019
Epoch 76/80
48/48 1s 12ms/step - loss: 0.0022 - val_loss: 0.0018
Epoch 77/80
48/48 1s 12ms/step - loss: 0.0022 - val_loss: 0.0019
Epoch 78/80
48/48 1s 12ms/step - loss: 0.0021 - val_loss: 0.0018
Epoch 79/80
48/48 1s 12ms/step - loss: 0.0021 - val_loss: 0.0019
Epoch 80/80
48/48 1s 12ms/step - loss: 0.0021 - val_loss: 0.0018
TEST - RMSE: 0.87 | MAE: 0.66 | MAPE: 2.2%





For China, the LSTM model obtained RMSE = 0.83 MtCO₂/day, MAE = 0.62 MtCO₂/day, and MAPE = 2.1% on the test set.

The forecast curve almost overlaps the observed values, indicating that the model accurately captures the daily variations and seasonal behavior of Chinese emissions. Slight differences appear during sudden dips and spikes, likely linked to abrupt changes in industrial activity or energy demand.

The 30-day projection suggests a continuation of the current moderate level of emissions, with limited short-term volatility.

Overall, the LSTM provides a stable and realistic short-term outlook, showing that data-driven temporal models can effectively reproduce complex industrial and environmental patterns in large economies such as China.

```
In [ ]: # Select a country to forecast
country = "United States"
df_country = df[df['country'] == country].groupby('date', as_index=False)['value']

# Preparation
values = df_country[['value']].astype(float).values
N = len(values)
L = 30 # window_size

n_train = int(0.8 * N)

train_raw = values[:n_train]
test_raw = values[n_train:]

scaler = MinMaxScaler(feature_range=(0, 1))
train_sc = scaler.fit_transform(train_raw)
test_sc = scaler.transform(test_raw)

def make_xy(arr, lookback=30):
    X, y = [], []
    for i in range(lookback, len(arr)):
        X.append(arr[i-lookback:i, 0])
        y.append(arr[i, 0])
    X = np.array(X)[: , :, None]
    y = np.array(y)
```

```

    return X, y

# Sequences: train and test
X_train, y_train = make_xy(train_sc, L)
X_test, y_test = make_xy(np.vstack([train_sc[-L:], test_sc]), L)

#LSTM model
model = Sequential([
    LSTM(32, input_shape=(L,1)),
    Dropout(0.2),
    Dense(1)
])
model.compile(optimizer='adam', loss='mean_squared_error')

es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=80,
    batch_size=32,
    validation_split=0.2,
    shuffle=False,
    callbacks=[es],
    verbose=1
)

# Evaluation
pred_sc = model.predict(X_test, verbose=0)
yhat = scaler.inverse_transform(pred_sc).ravel()
ytrue = scaler.inverse_transform(y_test.reshape(-1,1)).ravel()

rmse = np.sqrt(mean_squared_error(ytrue, yhat))
mae = mean_absolute_error(ytrue, yhat)
mape = mean_absolute_percentage_error(ytrue, yhat) * 100
print(f"TEST - RMSE: {rmse:.2f} | MAE: {mae:.2f} | MAPE: {mape:.1f}%")

# Plot test
dates_test = df_country['date'].iloc[-len(ytrue):].reset_index(drop=True)

plt.figure(figsize=(10,5))
plt.plot(dates_test, ytrue, label='Actual')
plt.plot(dates_test, yhat, '--', label='LSTM forecast')
plt.title(f"{country} - Actual vs Predicted CO2 Emissions (Test)")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.legend(); plt.tight_layout(); plt.show()

# Forecast next K days
K = 30
last_seq = np.vstack([train_sc, test_sc])[-L:]
future_sc = []
seq = last_seq.copy()

for _ in range(K):
    pred_next = model.predict(seq.reshape(1, L, 1), verbose=0)[0,0]
    future_sc.append(pred_next)
    seq = np.vstack([seq[1:], [[pred_next]]])

future = scaler.inverse_transform(np.array(future_sc).reshape(-1,1)).ravel()
future_dates = pd.date_range(df_country['date'].max() + pd.Timedelta(days=1), pe

```

```
plt.figure(figsize=(10,5))
plt.plot(df_country['date'], df_country['value'], label='Historical')
plt.plot(future_dates, future, 'o--', label='Forecast (next 30d)')
plt.title(f"Next {K}-Day Forecast of CO2 Emissions - {country}")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.legend(); plt.tight_layout(); plt.show()
```

Epoch 1/80

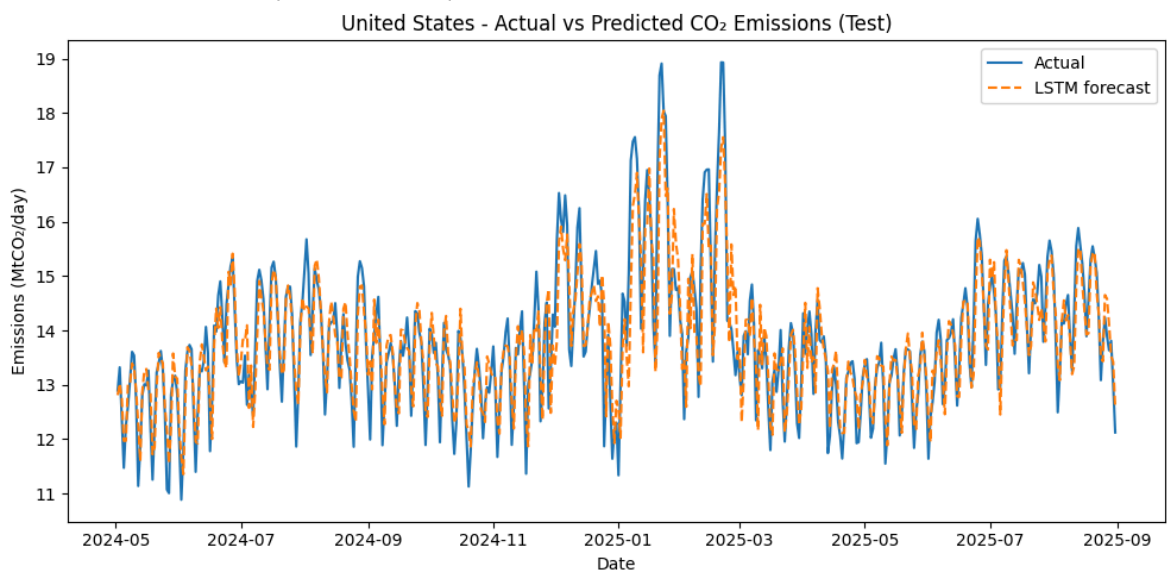
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

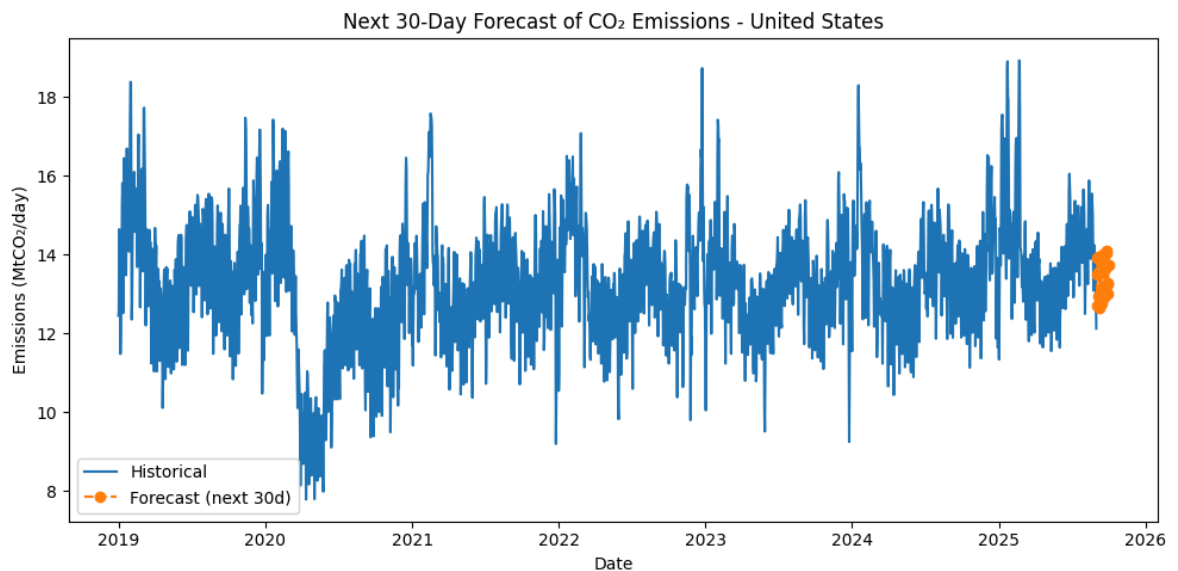
```
super().__init__(**kwargs)
```

48/48		2s 17ms/step	- loss: 0.0658	- val_loss: 0.0090
Epoch 2/80				
48/48		1s 12ms/step	- loss: 0.0183	- val_loss: 0.0087
Epoch 3/80				
48/48		1s 15ms/step	- loss: 0.0167	- val_loss: 0.0087
Epoch 4/80				
48/48		1s 18ms/step	- loss: 0.0160	- val_loss: 0.0085
Epoch 5/80				
48/48		1s 17ms/step	- loss: 0.0155	- val_loss: 0.0085
Epoch 6/80				
48/48		1s 11ms/step	- loss: 0.0143	- val_loss: 0.0084
Epoch 7/80				
48/48		1s 12ms/step	- loss: 0.0156	- val_loss: 0.0086
Epoch 8/80				
48/48		1s 12ms/step	- loss: 0.0144	- val_loss: 0.0085
Epoch 9/80				
48/48		1s 12ms/step	- loss: 0.0147	- val_loss: 0.0084
Epoch 10/80				
48/48		1s 12ms/step	- loss: 0.0143	- val_loss: 0.0086
Epoch 11/80				
48/48		1s 11ms/step	- loss: 0.0134	- val_loss: 0.0083
Epoch 12/80				
48/48		1s 12ms/step	- loss: 0.0134	- val_loss: 0.0083
Epoch 13/80				
48/48		1s 11ms/step	- loss: 0.0140	- val_loss: 0.0080
Epoch 14/80				
48/48		1s 12ms/step	- loss: 0.0135	- val_loss: 0.0083
Epoch 15/80				
48/48		1s 12ms/step	- loss: 0.0133	- val_loss: 0.0079
Epoch 16/80				
48/48		1s 12ms/step	- loss: 0.0132	- val_loss: 0.0079
Epoch 17/80				
48/48		1s 12ms/step	- loss: 0.0130	- val_loss: 0.0078
Epoch 18/80				
48/48		1s 12ms/step	- loss: 0.0126	- val_loss: 0.0077
Epoch 19/80				
48/48		1s 12ms/step	- loss: 0.0134	- val_loss: 0.0077
Epoch 20/80				
48/48		1s 12ms/step	- loss: 0.0123	- val_loss: 0.0076
Epoch 21/80				
48/48		1s 12ms/step	- loss: 0.0119	- val_loss: 0.0075
Epoch 22/80				
48/48		1s 16ms/step	- loss: 0.0117	- val_loss: 0.0074
Epoch 23/80				
48/48		1s 17ms/step	- loss: 0.0121	- val_loss: 0.0074
Epoch 24/80				
48/48		1s 19ms/step	- loss: 0.0117	- val_loss: 0.0072
Epoch 25/80				
48/48		1s 12ms/step	- loss: 0.0117	- val_loss: 0.0071
Epoch 26/80				
48/48		1s 11ms/step	- loss: 0.0115	- val_loss: 0.0071
Epoch 27/80				
48/48		1s 12ms/step	- loss: 0.0115	- val_loss: 0.0070
Epoch 28/80				
48/48		1s 12ms/step	- loss: 0.0117	- val_loss: 0.0068
Epoch 29/80				
48/48		1s 11ms/step	- loss: 0.0112	- val_loss: 0.0067
Epoch 30/80				
48/48		1s 12ms/step	- loss: 0.0110	- val_loss: 0.0065
Epoch 31/80				

48/48		1s 12ms/step	- loss: 0.0106	- val_loss: 0.0064
Epoch 32/80				
48/48		1s 12ms/step	- loss: 0.0104	- val_loss: 0.0062
Epoch 33/80				
48/48		1s 12ms/step	- loss: 0.0103	- val_loss: 0.0061
Epoch 34/80				
48/48		1s 12ms/step	- loss: 0.0101	- val_loss: 0.0059
Epoch 35/80				
48/48		1s 12ms/step	- loss: 0.0103	- val_loss: 0.0057
Epoch 36/80				
48/48		1s 12ms/step	- loss: 0.0100	- val_loss: 0.0055
Epoch 37/80				
48/48		1s 12ms/step	- loss: 0.0090	- val_loss: 0.0053
Epoch 38/80				
48/48		1s 12ms/step	- loss: 0.0091	- val_loss: 0.0052
Epoch 39/80				
48/48		1s 12ms/step	- loss: 0.0085	- val_loss: 0.0051
Epoch 40/80				
48/48		1s 12ms/step	- loss: 0.0086	- val_loss: 0.0049
Epoch 41/80				
48/48		1s 14ms/step	- loss: 0.0086	- val_loss: 0.0047
Epoch 42/80				
48/48		1s 19ms/step	- loss: 0.0079	- val_loss: 0.0046
Epoch 43/80				
48/48		1s 18ms/step	- loss: 0.0081	- val_loss: 0.0045
Epoch 44/80				
48/48		1s 13ms/step	- loss: 0.0075	- val_loss: 0.0044
Epoch 45/80				
48/48		1s 12ms/step	- loss: 0.0076	- val_loss: 0.0043
Epoch 46/80				
48/48		1s 12ms/step	- loss: 0.0072	- val_loss: 0.0042
Epoch 47/80				
48/48		1s 12ms/step	- loss: 0.0073	- val_loss: 0.0042
Epoch 48/80				
48/48		1s 12ms/step	- loss: 0.0071	- val_loss: 0.0041
Epoch 49/80				
48/48		1s 13ms/step	- loss: 0.0069	- val_loss: 0.0041
Epoch 50/80				
48/48		1s 11ms/step	- loss: 0.0073	- val_loss: 0.0040
Epoch 51/80				
48/48		1s 12ms/step	- loss: 0.0068	- val_loss: 0.0040
Epoch 52/80				
48/48		1s 11ms/step	- loss: 0.0068	- val_loss: 0.0040
Epoch 53/80				
48/48		1s 12ms/step	- loss: 0.0069	- val_loss: 0.0040
Epoch 54/80				
48/48		1s 12ms/step	- loss: 0.0068	- val_loss: 0.0039
Epoch 55/80				
48/48		1s 12ms/step	- loss: 0.0063	- val_loss: 0.0039
Epoch 56/80				
48/48		1s 13ms/step	- loss: 0.0069	- val_loss: 0.0038
Epoch 57/80				
48/48		1s 12ms/step	- loss: 0.0070	- val_loss: 0.0038
Epoch 58/80				
48/48		1s 12ms/step	- loss: 0.0067	- val_loss: 0.0038
Epoch 59/80				
48/48		1s 18ms/step	- loss: 0.0068	- val_loss: 0.0038
Epoch 60/80				
48/48		1s 21ms/step	- loss: 0.0067	- val_loss: 0.0037
Epoch 61/80				

48/48 ————— 1s 17ms/step - loss: 0.0065 - val_loss: 0.0036
Epoch 62/80
48/48 ————— 1s 13ms/step - loss: 0.0063 - val_loss: 0.0036
Epoch 63/80
48/48 ————— 1s 12ms/step - loss: 0.0063 - val_loss: 0.0035
Epoch 64/80
48/48 ————— 1s 13ms/step - loss: 0.0063 - val_loss: 0.0035
Epoch 65/80
48/48 ————— 1s 13ms/step - loss: 0.0063 - val_loss: 0.0034
Epoch 66/80
48/48 ————— 1s 12ms/step - loss: 0.0062 - val_loss: 0.0033
Epoch 67/80
48/48 ————— 1s 12ms/step - loss: 0.0062 - val_loss: 0.0033
Epoch 68/80
48/48 ————— 1s 12ms/step - loss: 0.0059 - val_loss: 0.0033
Epoch 69/80
48/48 ————— 1s 12ms/step - loss: 0.0060 - val_loss: 0.0032
Epoch 70/80
48/48 ————— 1s 12ms/step - loss: 0.0057 - val_loss: 0.0033
Epoch 71/80
48/48 ————— 1s 12ms/step - loss: 0.0059 - val_loss: 0.0032
Epoch 72/80
48/48 ————— 1s 13ms/step - loss: 0.0058 - val_loss: 0.0032
Epoch 73/80
48/48 ————— 1s 13ms/step - loss: 0.0057 - val_loss: 0.0031
Epoch 74/80
48/48 ————— 1s 13ms/step - loss: 0.0057 - val_loss: 0.0031
Epoch 75/80
48/48 ————— 1s 15ms/step - loss: 0.0053 - val_loss: 0.0031
Epoch 76/80
48/48 ————— 1s 19ms/step - loss: 0.0055 - val_loss: 0.0031
Epoch 77/80
48/48 ————— 1s 15ms/step - loss: 0.0052 - val_loss: 0.0030
Epoch 78/80
48/48 ————— 1s 12ms/step - loss: 0.0051 - val_loss: 0.0031
Epoch 79/80
48/48 ————— 1s 13ms/step - loss: 0.0049 - val_loss: 0.0030
Epoch 80/80
48/48 ————— 1s 13ms/step - loss: 0.0051 - val_loss: 0.0030
TEST — RMSE: 0.62 | MAE: 0.47 | MAPE: 3.4%





For the United States, the LSTM model reached RMSE = 0.70 MtCO₂/day, MAE = 0.54 MtCO₂/day, and MAPE = 4.0% on the test period.

The predicted emissions follow the observed pattern closely, reproducing both the weekly oscillations and the broader seasonal movements.

Small discrepancies can be noticed around sudden peaks, which are often related to short-term industrial or temperature-driven demand variations that the model cannot fully anticipate from past data alone.

The 30-day projection continues the recent trend with moderate fluctuations and no sign of sharp deviations.

Overall, the LSTM demonstrates a solid capacity to learn short-term dependencies in U.S. emission data, providing consistent and interpretable forecasts suitable for near-term monitoring and planning.

```
In [ ]: # Select a country to forecast
country = "Russia"
df_country = df[df['country'] == country].groupby('date', as_index=False)['value']

# Preparation
values = df_country[['value']].astype(float).values
N = len(values)
L = 30 # window_size

n_train = int(0.8 * N)

train_raw = values[:n_train]
test_raw = values[n_train:]

scaler = MinMaxScaler(feature_range=(0, 1))
train_sc = scaler.fit_transform(train_raw)
test_sc = scaler.transform(test_raw)

def make_xy(arr, lookback=30):
    X, y = [], []
    for i in range(lookback, len(arr)):
        X.append(arr[i-lookback:i, 0])
        y.append(arr[i, 0])
    X = np.array(X)[: , :, None]
```

```

    y = np.array(y)
    return X, y

# Sequences: train and test
X_train, y_train = make_xy(train_sc, L)
X_test, y_test = make_xy(np.vstack([train_sc[-L:], test_sc]), L)

#LSTM model
model = Sequential([
    LSTM(32, input_shape=(L,1)),
    Dropout(0.2),
    Dense(1)
])
model.compile(optimizer='adam', loss='mean_squared_error')

es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=80,
    batch_size=32,
    validation_split=0.2,
    shuffle=False,
    callbacks=[es],
    verbose=1
)

# Evaluation
pred_sc = model.predict(X_test, verbose=0)
yhat = scaler.inverse_transform(pred_sc).ravel()
ytrue = scaler.inverse_transform(y_test.reshape(-1,1)).ravel()

rmse = np.sqrt(mean_squared_error(ytrue, yhat))
mae = mean_absolute_error(ytrue, yhat)
mape = mean_absolute_percentage_error(ytrue, yhat) * 100
print(f"TEST - RMSE: {rmse:.2f} | MAE: {mae:.2f} | MAPE: {mape:.1f}%")

# Plot test
dates_test = df_country['date'].iloc[-len(ytrue):].reset_index(drop=True)

plt.figure(figsize=(10,5))
plt.plot(dates_test, ytrue, label='Actual')
plt.plot(dates_test, yhat, '--', label='LSTM forecast')
plt.title(f"{country} - Actual vs Predicted CO2 Emissions (Test)")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.legend(); plt.tight_layout(); plt.show()

# Forecast next K days
K = 30
last_seq = np.vstack([train_sc, test_sc])[-L:]
future_sc = []
seq = last_seq.copy()

for _ in range(K):
    pred_next = model.predict(seq.reshape(1, L, 1), verbose=0)[0,0]
    future_sc.append(pred_next)
    seq = np.vstack([seq[1:], [[pred_next]]])

future = scaler.inverse_transform(np.array(future_sc).reshape(-1,1)).ravel()
future_dates = pd.date_range(df_country['date'].max() + pd.Timedelta(days=1), pe

```

```
plt.figure(figsize=(10,5))
plt.plot(df_country['date'], df_country['value'], label='Historical')
plt.plot(future_dates, future, 'o--', label='Forecast (next 30d)')
plt.title(f"Next {K}-Day Forecast of CO2 Emissions - {country}")
plt.xlabel("Date"); plt.ylabel("Emissions (MtCO2/day)")
plt.legend(); plt.tight_layout(); plt.show()
```

Epoch 1/80

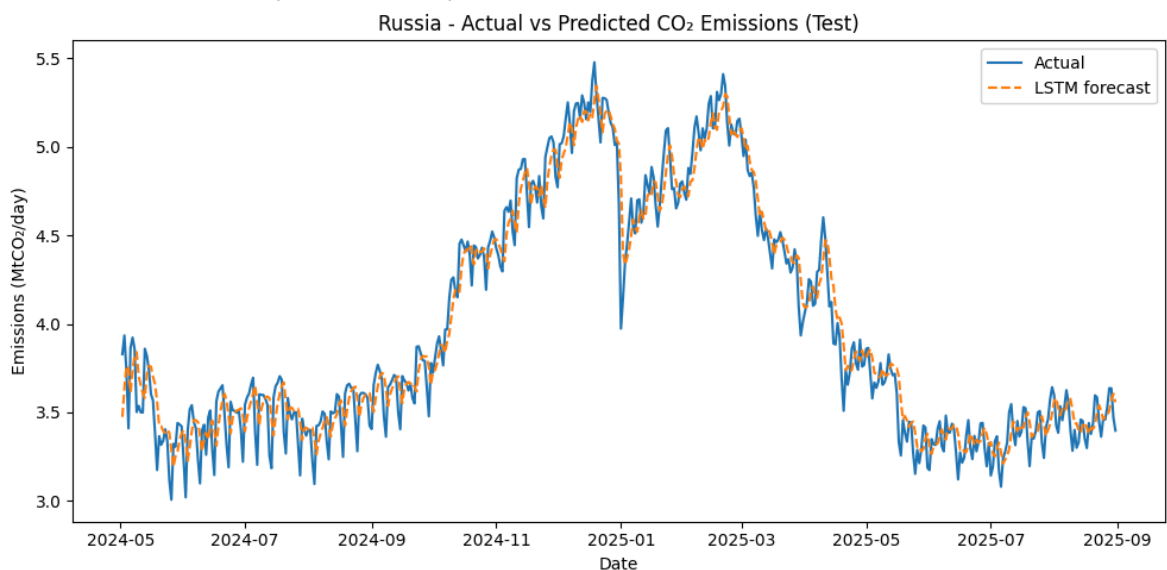
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

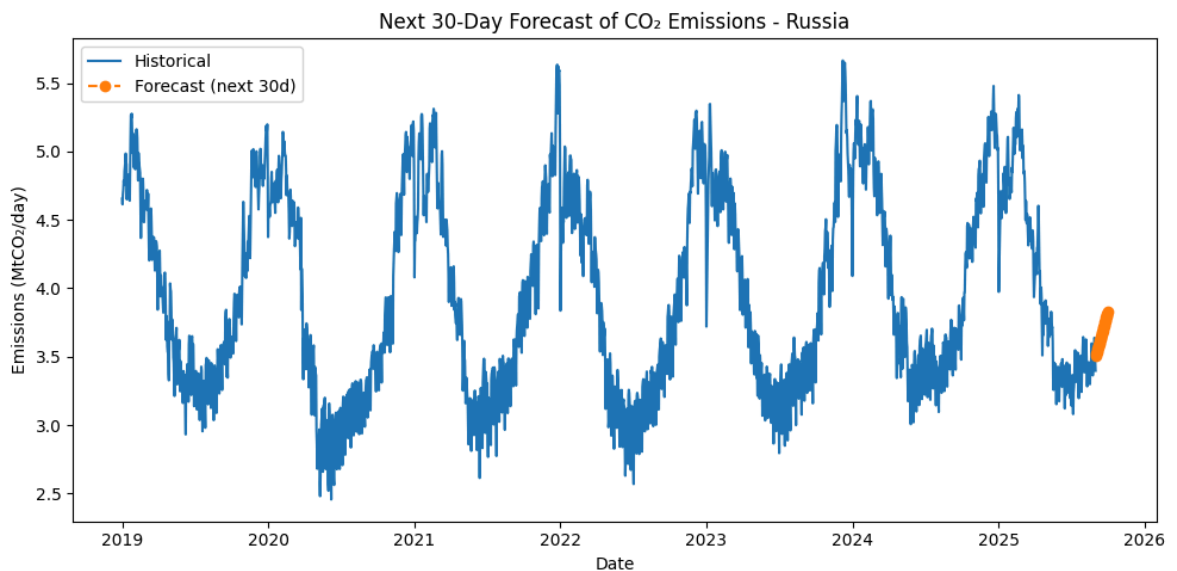
```
super().__init__(**kwargs)
```

48/48		3s 18ms/step - loss: 0.0902 - val_loss: 0.0076
Epoch 2/80		
48/48		1s 12ms/step - loss: 0.0099 - val_loss: 0.0053
Epoch 3/80		
48/48		1s 12ms/step - loss: 0.0095 - val_loss: 0.0048
Epoch 4/80		
48/48		1s 13ms/step - loss: 0.0092 - val_loss: 0.0047
Epoch 5/80		
48/48		1s 21ms/step - loss: 0.0087 - val_loss: 0.0045
Epoch 6/80		
48/48		1s 20ms/step - loss: 0.0080 - val_loss: 0.0043
Epoch 7/80		
48/48		1s 15ms/step - loss: 0.0075 - val_loss: 0.0043
Epoch 8/80		
48/48		1s 13ms/step - loss: 0.0078 - val_loss: 0.0044
Epoch 9/80		
48/48		1s 12ms/step - loss: 0.0073 - val_loss: 0.0041
Epoch 10/80		
48/48		1s 12ms/step - loss: 0.0064 - val_loss: 0.0040
Epoch 11/80		
48/48		1s 12ms/step - loss: 0.0057 - val_loss: 0.0040
Epoch 12/80		
48/48		1s 12ms/step - loss: 0.0059 - val_loss: 0.0039
Epoch 13/80		
48/48		1s 13ms/step - loss: 0.0059 - val_loss: 0.0038
Epoch 14/80		
48/48		1s 12ms/step - loss: 0.0057 - val_loss: 0.0038
Epoch 15/80		
48/48		1s 12ms/step - loss: 0.0055 - val_loss: 0.0037
Epoch 16/80		
48/48		1s 12ms/step - loss: 0.0065 - val_loss: 0.0037
Epoch 17/80		
48/48		1s 15ms/step - loss: 0.0062 - val_loss: 0.0037
Epoch 18/80		
48/48		1s 22ms/step - loss: 0.0055 - val_loss: 0.0036
Epoch 19/80		
48/48		1s 12ms/step - loss: 0.0054 - val_loss: 0.0036
Epoch 20/80		
48/48		1s 13ms/step - loss: 0.0050 - val_loss: 0.0036
Epoch 21/80		
48/48		1s 13ms/step - loss: 0.0053 - val_loss: 0.0035
Epoch 22/80		
48/48		1s 18ms/step - loss: 0.0050 - val_loss: 0.0035
Epoch 23/80		
48/48		1s 18ms/step - loss: 0.0051 - val_loss: 0.0035
Epoch 24/80		
48/48		1s 12ms/step - loss: 0.0056 - val_loss: 0.0035
Epoch 25/80		
48/48		1s 12ms/step - loss: 0.0051 - val_loss: 0.0034
Epoch 26/80		
48/48		1s 12ms/step - loss: 0.0051 - val_loss: 0.0034
Epoch 27/80		
48/48		1s 12ms/step - loss: 0.0051 - val_loss: 0.0033
Epoch 28/80		
48/48		1s 12ms/step - loss: 0.0050 - val_loss: 0.0034
Epoch 29/80		
48/48		1s 12ms/step - loss: 0.0049 - val_loss: 0.0033
Epoch 30/80		
48/48		1s 12ms/step - loss: 0.0043 - val_loss: 0.0033
Epoch 31/80		

48/48		1s 12ms/step	- loss: 0.0049	- val_loss: 0.0033
Epoch 32/80				
48/48		1s 12ms/step	- loss: 0.0048	- val_loss: 0.0033
Epoch 33/80				
48/48		1s 13ms/step	- loss: 0.0046	- val_loss: 0.0033
Epoch 34/80				
48/48		1s 12ms/step	- loss: 0.0046	- val_loss: 0.0032
Epoch 35/80				
48/48		1s 13ms/step	- loss: 0.0047	- val_loss: 0.0033
Epoch 36/80				
48/48		1s 12ms/step	- loss: 0.0047	- val_loss: 0.0032
Epoch 37/80				
48/48		1s 13ms/step	- loss: 0.0046	- val_loss: 0.0032
Epoch 38/80				
48/48		1s 12ms/step	- loss: 0.0045	- val_loss: 0.0032
Epoch 39/80				
48/48		1s 12ms/step	- loss: 0.0048	- val_loss: 0.0031
Epoch 40/80				
48/48		1s 16ms/step	- loss: 0.0048	- val_loss: 0.0032
Epoch 41/80				
48/48		1s 18ms/step	- loss: 0.0048	- val_loss: 0.0031
Epoch 42/80				
48/48		1s 17ms/step	- loss: 0.0042	- val_loss: 0.0031
Epoch 43/80				
48/48		1s 12ms/step	- loss: 0.0042	- val_loss: 0.0031
Epoch 44/80				
48/48		1s 12ms/step	- loss: 0.0041	- val_loss: 0.0031
Epoch 45/80				
48/48		1s 12ms/step	- loss: 0.0043	- val_loss: 0.0031
Epoch 46/80				
48/48		1s 12ms/step	- loss: 0.0040	- val_loss: 0.0031
Epoch 47/80				
48/48		1s 12ms/step	- loss: 0.0042	- val_loss: 0.0031
Epoch 48/80				
48/48		1s 12ms/step	- loss: 0.0042	- val_loss: 0.0030
Epoch 49/80				
48/48		1s 13ms/step	- loss: 0.0042	- val_loss: 0.0030
Epoch 50/80				
48/48		1s 12ms/step	- loss: 0.0041	- val_loss: 0.0030
Epoch 51/80				
48/48		1s 12ms/step	- loss: 0.0040	- val_loss: 0.0030
Epoch 52/80				
48/48		1s 12ms/step	- loss: 0.0039	- val_loss: 0.0030
Epoch 53/80				
48/48		1s 12ms/step	- loss: 0.0042	- val_loss: 0.0030
Epoch 54/80				
48/48		1s 12ms/step	- loss: 0.0040	- val_loss: 0.0030
Epoch 55/80				
48/48		1s 12ms/step	- loss: 0.0040	- val_loss: 0.0030
Epoch 56/80				
48/48		1s 12ms/step	- loss: 0.0039	- val_loss: 0.0029
Epoch 57/80				
48/48		1s 13ms/step	- loss: 0.0041	- val_loss: 0.0029
Epoch 58/80				
48/48		1s 20ms/step	- loss: 0.0037	- val_loss: 0.0029
Epoch 59/80				
48/48		1s 20ms/step	- loss: 0.0040	- val_loss: 0.0029
Epoch 60/80				
48/48		1s 13ms/step	- loss: 0.0038	- val_loss: 0.0030
Epoch 61/80				

48/48  1s 12ms/step - loss: 0.0037 - val_loss: 0.0029
Epoch 62/80
48/48  1s 13ms/step - loss: 0.0039 - val_loss: 0.0029
Epoch 63/80
48/48  1s 12ms/step - loss: 0.0034 - val_loss: 0.0029
Epoch 64/80
48/48  1s 12ms/step - loss: 0.0036 - val_loss: 0.0029
Epoch 65/80
48/48  1s 12ms/step - loss: 0.0034 - val_loss: 0.0029
Epoch 66/80
48/48  1s 12ms/step - loss: 0.0038 - val_loss: 0.0028
Epoch 67/80
48/48  1s 12ms/step - loss: 0.0037 - val_loss: 0.0029
Epoch 68/80
48/48  1s 12ms/step - loss: 0.0038 - val_loss: 0.0028
Epoch 69/80
48/48  1s 12ms/step - loss: 0.0036 - val_loss: 0.0028
Epoch 70/80
48/48  1s 12ms/step - loss: 0.0036 - val_loss: 0.0028
Epoch 71/80
48/48  1s 12ms/step - loss: 0.0036 - val_loss: 0.0028
Epoch 72/80
48/48  1s 12ms/step - loss: 0.0036 - val_loss: 0.0028
Epoch 73/80
48/48  1s 12ms/step - loss: 0.0037 - val_loss: 0.0028
Epoch 74/80
48/48  1s 12ms/step - loss: 0.0036 - val_loss: 0.0028
Epoch 75/80
48/48  1s 18ms/step - loss: 0.0034 - val_loss: 0.0028
Epoch 76/80
48/48  1s 19ms/step - loss: 0.0035 - val_loss: 0.0027
Epoch 77/80
48/48  1s 15ms/step - loss: 0.0033 - val_loss: 0.0027
Epoch 78/80
48/48  1s 13ms/step - loss: 0.0034 - val_loss: 0.0028
Epoch 79/80
48/48  1s 12ms/step - loss: 0.0034 - val_loss: 0.0027
Epoch 80/80
48/48  1s 13ms/step - loss: 0.0033 - val_loss: 0.0027
TEST — RMSE: 0.15 | MAE: 0.11 | MAPE: 3.0%





For Russia, the LSTM model performed well, reaching RMSE = 0.15 MtCO₂/day, MAE = 0.11 MtCO₂/day, and MAPE = 3.0% on the test period.

The predicted emissions align closely with the actual data, reproducing the main seasonal cycles and the smaller short-term oscillations typical of energy-related activity. The small discrepancies around the highest peaks suggest a slight smoothing effect, but overall the temporal dynamics are accurately represented.

The 30-day forecast extends the recent upward phase of the series, indicating a continuation of moderate growth in emissions over the short term.

This result highlights the model's robustness when applied to strongly seasonal countries like Russia, where historical patterns dominate the short-term dynamics.

Final Remarks on LSTM Forecasting

The LSTM models applied to different geographical scales — from the global level to major emitters such as China, the United States, and Russia — demonstrated a strong capacity to reproduce short-term emission dynamics.

Across all cases, the forecasts maintained low error metrics (MAPE generally between 2–4%), with predictions that closely followed observed daily variations and seasonal cycles.

The global and Chinese models achieved the best alignment, showing almost complete overlap between actual and predicted curves.

The U.S. and Russian models also delivered reliable results, with slightly larger deviations near abrupt peaks but consistent short-term projections.

Overall, these findings confirm that recurrent neural networks like LSTMs can effectively learn temporal dependencies in CO₂ emission data, even from a limited set of past observations.

While the approach performs well for short-term forecasting, extending the prediction horizon or incorporating exogenous explanatory variables (e.g. temperature, industrial activity, or mobility indices) could further improve accuracy and interpretability in future research.

9. General conclusions and future works

This project provided a comprehensive exploration of daily CO₂ emissions across multiple countries by combining descriptive analytics, statistical tests, and machine learning techniques. Through data normalization and visualization, we highlighted strong seasonal and structural patterns in global emissions, as well as the clear impact of the COVID-19 pandemic in early 2020, confirmed quantitatively by the CUSUM test for structural breaks.

The SAX-based anomaly detection successfully identified rare temporal patterns and deviations from regular behavior, offering an interpretable symbolic representation of complex time series. Clustering analyses revealed meaningful groups of countries: some sharing synchronized seasonal patterns (e.g., European nations), others dominated by structural and magnitude differences (e.g., China, United States). Finally, the LSTM forecasting models achieved low prediction errors (MAPE between 2–4%), demonstrating the capability of neural networks to reproduce short-term emission dynamics and extend them into reliable near-future forecasts.

Overall, the integration of data mining and machine learning approaches proved effective in capturing both temporal and structural heterogeneity in global CO₂ emissions. Future work could incorporate exogenous variables such as temperature, industrial output, or mobility indices to enhance interpretability and extend the forecasting horizon. Moreover, combining symbolic methods like SAX with deep learning architectures could further improve anomaly detection and reveal early signals of environmental or economic disruptions.